A Project Report on

"AUTOCAB BOOKING SYSTEM"

Submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Application

AKS UNIVERSITY, SATNA

BCA 5th Semester

Submitted by

Rahul Pal (B2492R10400159)

Balendra Singh (B2492R10400158)

Under The Guidance of

Dr. Pinki Sharma

(Assistant Professor)

Approval by

Dr. Pramod Singh

(Head CS/IT)

Department of Computer Science

Faculty of Computer Application & Information Technology and Sciences

AKS University, Satna (M.P.)

CERTIFICATE

This certifies that the project report entitled "AUTOCAB BOOKING SYSTEM" submitted in partial fulfillment of the requirement for the degree of Bachelor of Computer Application in July-Dec 2026, AKS University, Satna is a Bonafide project work carried out by Rahul Pal, B2492R10400159 and Balendra Singh, B2492R10400158, under my supervision. The subject of the project report has been approved by the supervisor. This is also to certify that it is his/her original work and no part of this project report has been submitted for any other degree/diploma.

All the assistance help received during the course of the investigation has been duly acknowledged.

1. I am satisfied that the report presented by Rahul Pal and Balendra Singh is worthy of consideration for the award of the degree.

2. I certify:

i) That he/she pursued the prescribed course for the project.

ii) That he/she bears good moral character.

Place: AKSU, Satna

Date: …/…./……

…….

Supervisor

(Dr. Pinki Sharma)

(Assistant Professor)

…….

Head of Department

Dr. Pramod Singh

…….

External Signature

SELF DECLARATION

I hereby declare that the work presented in this project entitled "AUTOCAB BOOKING SYSTEM" towards the partial fulfillment of the requirement for the award of a Degree in BCA in the Department of Computer Science, AKS University, Satna (M.P.) is an authentic record of my work.

I have not submitted the matter embodied in the project for the award of any other degree or diploma to any other institute or university.

……..
Signature of Candidate
Rahul Pal (B2492R10400159)

……..
Signature of Candidate
Balendra Singh (B2492R10400158)

ACKNOWLEDGEMENT

It is a great pleasure for me to take this opportunity to express my sincere thanks and appreciation to Dr. Pramod Singh, Head of the Department of CS, AKS University, Satna (M.P.).

I consider myself lucky to have got such a great project. This project will be an asset to my profile.

At this moment of accomplishment, first of all, I pay homage to my guide, Dr. Pinki Sharma (Assistant Professor), AKS University, Satna (M.P.). This work would not have been possible without her guidance, support and encouragement. Under her guidance I successfully overcame many difficulties and learned a lot.

I am deeply and forever indebted to my parents for their love, support and encouragement throughout my entire life.

ABSTRACT

The “AutoCab Booking System” is a comprehensive web-based platform designed to automate cab booking and fleet operations. As urban populations grow, the demand for efficient, transparent and reliable transportation has increased significantly, yet many local transport businesses still depend on telephone calls and paper registers, which cause over-booking, fare disputes and the absence of reliable records. This project proposes a centralised digital solution that connects passengers and drivers and gives administrative control to the fleet owner.

The system is organised into three portals: the Passenger Portal, the Driver Portal and the Admin Dashboard. Passengers can register, log in, view the cabs that are currently available in the Sedan, SUV, Luxury and EV categories, see an estimated fare based on distance and the vehicle rate, and book a ride instantly. Drivers can manage their availability status and the trips assigned to them. The administrator can register drivers, add cabs, manage users, view all bookings and monitor revenue. Every booking follows a defined life cycle – Pending, Accepted, In Progress, Completed or Cancelled – and the cab is automatically marked “On Ride” when booked and “Available” again when the trip ends, which completely removes over-booking.

The back end is built with the FastAPI framework of Python and a SQLite database accessed through the SQLAlchemy ORM, with Pydantic schemas for validation. Security is provided by JWT (JSON Web Token) authentication, bcrypt password hashing and role-based authorisation. The front end uses HTML, CSS and JavaScript, and a React/TypeScript prototype demonstrates the complete role-based dashboards. The database contains four related tables – users, drivers, cabs and bookings.

The AutoCab system eliminates manual scheduling errors, provides fare transparency, keeps a complete digital record of trips and revenue, and offers a low-cost, scalable infrastructure for small and medium transport service providers. It can be extended in future with GPS tracking, online payment, AI-based demand analytics and mobile applications.

TABLE OF CONTENTS

1. INTRODUCTION....1
2. OBJECTIVE OF THE PROJECT....3
3. SCOPE OF THE PROJECT....4
4. DEFINITION OF PROBLEM....5
5. BENEFIT OF THE PROJECT....7
6. MAIN MODULES OF THE PROJECT....8
7. TECHNICAL OVERVIEW....11
8. SYSTEM CONFIGURATION....14
9. BLOCK DIAGRAM....18
10. DATA FLOW DIAGRAMS (DFD)....20
11. ENTITY RELATIONSHIP (ER) DIAGRAM....23
12. OUTPUT SCREENS....27
13. PROJECT EXPLANATION....33
14. CODING (SAMPLE WITH EXPLANATION)....40
15. SOFTWARE TESTING....70
16. CONCLUSION....75
17. BIBLIOGRAPHY / REFERENCES....76

1. INTRODUCTION

1.1 Background of the Problem

Urban transportation has changed rapidly over the last decade. As cities grow, people depend more and more on taxis and cabs for daily travel to offices, railway stations, airports, hospitals and markets. Large aggregator platforms have set a high expectation: a passenger should be able to see which vehicles are available, know the fare before the ride starts, and follow the progress of the booking from a phone or a computer.

However, a very large number of small and medium transport businesses still work in the traditional way. Bookings are taken over telephone calls, entries are written in paper registers, drivers are informed by phone, and the owner calculates the fare manually after the trip. This traditional method creates several practical difficulties. The same cab can be promised to two customers at the same time (over-booking), nobody knows in real time which vehicle is free, fare disputes arise because there is no fixed method of calculation, and the owner has no reliable record of how many trips a driver has completed or how much revenue the fleet has produced.

The **AutoCab Booking System** has been designed to remove these difficulties. It is a web-based platform that connects passengers, drivers and fleet administrators through a single centralised application. The back end is a RESTful API developed with the **FastAPI** framework of Python and a **SQLite** database accessed through the **SQLAlchemy** ORM. The front end is built with HTML, CSS and JavaScript and communicates with the API using JSON over HTTP. Security is handled with **JWT (JSON Web Token)** authentication and **bcrypt** password hashing.

1.2 Overview of the System

AutoCab supports three types of users. A **passenger** can register, log in, look at the cabs that are currently available (filtered by Sedan, SUV, Luxury or EV category), enter pickup and destination locations, see the estimated fare and confirm a booking. A **driver** can log in and manage availability and the trips assigned to him or her. An **administrator** has complete control over the fleet: registering drivers, adding cabs, viewing all users and bookings and monitoring revenue.

Every booking passes through a defined life cycle: **Pending → Accepted → In Progress → Completed**, or **Cancelled** at any time before completion. When a booking is created the selected cab is automatically marked as "On Ride" so that it cannot be booked again; when the booking is completed or cancelled the cab is released and becomes "Available" again. This automatic synchronisation between the booking table and the cab table is the heart of the system because it removes the over-booking problem completely.

1.3 Need for the Project

Small fleet owners cannot afford to build or rent large commercial dispatch software. At the same time their customers are used to the convenience of online booking. AutoCab fills this gap by using only open-source technologies, so that the cost of development, deployment and maintenance stays very low, while the quality of service (transparency, speed and record keeping) becomes comparable to that of bigger aggregators.

1.4 Organisation of the Report

This report is organised in the sequence prescribed by the department. Chapters 2 to 5 describe the objectives, scope, problem definition and benefits of the project. Chapter 6 explains the main modules, Chapter 7 gives the technical overview and Chapter 8 lists the system configuration. Chapters 9 to 11 present the system design through the block diagram, data flow diagrams and entity relationship diagram. Chapter 12 shows the output screens, Chapter 13 explains the working of the project in detail, and Chapter 14 contains the source code with explanation. Chapter 15 covers software testing, Chapter 16 gives the conclusion and future scope, and Chapter 17 lists the references.

2. OBJECTIVE OF THE PROJECT

The main aim of the project is to automate cab booking and fleet management so that manual scheduling errors are eliminated and passengers, drivers and owners work on the same, always up-to-date information. The specific objectives are listed below.

1. **To develop a secure multi-role web portal** for passengers, drivers and administrators, where every user sees only the functions that belong to his or her role.
2. **To implement an automated fare estimation engine** that calculates the fare from the vehicle category and the travel distance, so that the passenger knows the amount before confirming the booking.
3. **To provide real-time status information** about cab availability and about the progress of every booking (Pending, Accepted, In Progress, Completed, Cancelled).
4. **To create a centralised administrative interface** for fleet oversight: driver registration, cab registration, user management, booking log and revenue monitoring.
5. **To ensure performance and data integrity** by using the FastAPI framework, Pydantic validation and the SQLAlchemy ORM with proper primary-key and foreign-key relationships.
6. **To protect user data** through JWT based authentication, role based authorisation and one-way bcrypt hashing of passwords.
7. **To build a scalable and low-cost solution** that can run on an ordinary web server and can later be extended with GPS tracking, online payment and mobile applications.

2.1 Expected Outcomes

- Over-booking is prevented because a cab that has been booked is immediately removed from the list of available cabs.
- Fare disputes are reduced because the fare is shown and stored at the time of booking.
- The owner gets a complete, searchable history of all bookings, drivers and vehicles.
- Passengers get a simple interface that needs only basic web or smartphone knowledge.

3. SCOPE OF THE PROJECT

3.1 Scope of Work

The project covers the development of a RESTful API back end and a responsive web front end for a cab booking and fleet management system. The following areas are included in the scope of the present work.

- **User management** – registration, login and profile retrieval for the roles user (passenger), driver and admin.
- **Secure authentication** – JWT access tokens with 24-hour validity, bcrypt password hashing and role-protected API routes.
- **Vehicle categorisation** – Sedan, SUV, Luxury and EV categories, with filtering of available cabs by category.
- **Automated fare estimation** – fare calculated from distance and the per-kilometre rate of the selected vehicle category.
- **Booking life cycle** – creation of a booking, status updates (Pending, Accepted, In Progress, Completed, Cancelled) and automatic release of the cab.
- **Driver and fleet management** – registering drivers, adding cabs, assigning a driver to a cab and changing driver status (Available, Busy, Offline).
- **Administrative reporting** – list of all users, all bookings and system statistics such as total revenue and number of completed rides.

3.2 Limitations of the Present Version

The following items are intentionally kept outside the present scope and are mentioned in the Future Scope section of the report: live GPS tracking on a map, online payment gateway, automatic driver assignment by nearest location, e-mail or SMS notifications, and native mobile applications. In the present version the distance used for fare estimation is entered or estimated by the user instead of being calculated from a map service.

3.3 Target Users

The system is intended for small and medium cab operators, local taxi associations, corporate transport desks and institutions that manage their own vehicles, as well as for the passengers of such services.

4. DEFINITION OF PROBLEM

4.1 Problems in the Existing Manual System

A study of how local cab services work shows that most of them depend on telephone calls and handwritten registers. The main problems of this existing system are as follows.

Table 1: Problems of the manual system and the AutoCab solution

Problem in manual system	Effect on business	AutoCab solution
Bookings taken by phone call and written in a register	Entries are lost, duplicated or written wrongly	Every booking is stored in the bookings table with a unique ID
No real-time view of vehicle availability	The same cab is promised to two customers (over-booking)	Cab is marked "On Ride" at booking time and removed from the available list
Fare decided verbally after the trip	Disputes between driver and passenger	Fare is calculated before booking and saved with the booking
No record of driver performance	Owner cannot judge efficiency or pay incentives fairly	Driver profile, status and trip history are maintained
Maintenance of vehicles is not tracked	Vehicles break down unexpectedly	Cab availability can be set to Maintenance by the admin
Revenue calculated manually at month end	Calculation errors and delay in decisions	Admin dashboard shows total revenue from completed rides
No protection of customer data	Privacy risk, misuse of phone numbers	Hashed passwords, JWT tokens and role-based access

4.2 Problem Statement

"To design and develop a secure, web-based cab booking and fleet management system that allows passengers to find available cabs, estimate the fare and book a ride instantly, allows drivers to manage their availability and trips, and gives the fleet owner a central dashboard to control drivers, vehicles, bookings and revenue, while preventing over-booking and fare disputes."

4.3 Constraints Considered

- The solution must use free and open-source tools so that it is affordable for small operators.
- The solution must run on a normal computer or a low-cost web server (Intel i3, 4 GB RAM).
- The user interface must be simple enough for users with only basic web knowledge.
- The database must be light, file based and easy to back up.

5. BENEFIT OF THE PROJECT

5.1 Benefits for Passengers

- The passenger can see only those cabs that are actually available, filtered by category.
- The fare is known in advance, so there is no surprise at the end of the trip.
- Booking can be done at any time from any device that has a web browser.
- The passenger can see the booking history and the current status of every ride.

5.2 Benefits for Drivers

- Drivers can change their status to Available, Busy or Offline according to their working hours.
- Assigned trips and passenger details are available in one place instead of phone calls.
- Completed rides are counted automatically, which makes earnings and performance transparent.

5.3 Benefits for the Fleet Owner / Administrator

- Complete control of users, drivers, cabs and bookings from a single dashboard.
- Automatic revenue calculation from completed bookings.
- Reduction of paper work and of manual errors.
- Better utilisation of vehicles because availability is always up to date.
- A digital platform that allows a small operator to compete with large aggregators.

5.4 Technical and Economic Benefits

Since FastAPI, SQLAlchemy, SQLite and the front-end technologies are open source, there is no licence cost. SQLite stores the entire database in a single file, which makes installation and backup very easy. FastAPI automatically generates interactive API documentation (Swagger UI), which helps in testing and in future maintenance. The layered code structure (routes, schemas, CRUD functions, models) makes the application easy to extend.

6. MAIN MODULES OF THE PROJECT

The system is divided into functional modules. Each module is implemented by API routes in `main.py`, functions in `crud.py` and pages or components in the front end, as shown below.

Table 2: Modules and their implementation files

Module	Back-end files	Front-end files
Authentication Module	auth.py, main.py (/api/auth/*)	login.html, register.html, script.js
User / Passenger Module	main.py, crud.py (users, bookings)	index.html, booking.html, dashboard.html
Driver Management Module	main.py (/api/drivers/*), crud.py	Driver dashboard (React prototype)
Fleet (Cab) Management Module	main.py (/api/cabs/*), crud.py	booking.html (cab list), Admin dashboard
Booking Life Cycle Module	main.py (/api/bookings/*), crud.py	booking.html, dashboard.html
Admin Module	auth.py (require_admin), main.py	Admin dashboard (React prototype)

6.1 Authentication Module

This module controls who can enter the system. A new user registers with name, e-mail, phone, password and role. The password is never stored in plain text; `get_password_hash()` in `auth.py` converts it to a bcrypt hash first. At login, `verify_password()` checks the entered password against the stored hash and `create_access_token()` generates a signed JWT with the user e-mail in the "sub" claim and an expiry of 24 hours. The browser keeps this token in localStorage and sends it in the `Authorization: Bearer` header.

Protected routes use `get_current_user`, which decodes the token and raises HTTP 401 if it is missing, invalid or expired. Admin-only routes also use `require_admin`, which raises HTTP 403 for non-admin users.

6.2 User / Passenger Module

The passenger module is the most frequently used part of the system. It contains the home page that shows the vehicle categories, the booking page that lists the available cabs with a category filter, a trip form for pickup and destination, and a dashboard that shows "My Recent Bookings". The route `GET /api/cabs/available` returns only those cabs whose availability is "Available", and accepts an optional `cab_type` parameter for filtering. The route `GET /api/bookings/my` returns the bookings of the logged-in user in descending order of date.

6.3 Driver Management Module

This module maintains the driver profiles and driver status. A driver record contains name, phone number, a unique licence number and a status which can be Available, Busy or Offline. The administrator registers a driver through `POST /api/drivers`. The status of a driver is changed through `PUT /api/drivers/{driver_id}/status`. In the React dashboard prototype, the driver portal shows incoming ride requests, allows the driver to accept a request, start the trip and complete it, and summarises earnings and completed rides.

6.4 Fleet (Cab) Management Module

Every cab has a registration number (unique), a category (Sedan, SUV, Luxury or EV), an availability state (Available, On Ride or Maintenance) and an optional link to a driver through the foreign key `driver_id`. The administrator adds cabs through `POST /api/cabs`. The module also supports changing the availability of a cab, which is used when a vehicle is sent for maintenance.

6.5 Booking Life Cycle Module

This module manages the journey of a ride from request to completion. When the passenger confirms a booking, `create_booking()` stores the pickup location, destination and fare with the status "Pending" and sets the chosen cab to "On Ride". The status can then be updated through `PUT /api/bookings/{booking_id}/status`. If the new status is "Completed" or "Cancelled", the cab is automatically changed back to "Available". All five statuses and their transitions are explained in Chapter 13.

6.6 Admin Module

The admin module gives the owner a high-level view of the fleet. Through `GET /api/users` and `GET /api/bookings/all` the administrator can see every registered user and every booking. The React dashboard prototype adds an overview tab with statistics (total users, total cabs, total bookings, total revenue), tabs for users, drivers, cabs and bookings, and forms for registering a new driver and adding a new cab.

7. TECHNICAL OVERVIEW

The project is developed using the following languages, frameworks and tools. A brief description of each technology and the reason for choosing it is given below.

Table 3: Technologies used in AutoCab

Layer	Technology	Purpose in the project
Programming language	Python 3.10+	Back-end logic, API routes, database access
Web framework	FastAPI	Fast REST API, automatic validation and Swagger documentation
ASGI server	Uvicorn	Runs the FastAPI application
ORM	SQLAlchemy 2.x	Maps Python classes to database tables
Validation	Pydantic 2.x	Request and response schemas, e-mail validation
Database	SQLite	Light file-based relational database (autocab.db)
Security	python-jose, passlib[bcrypt]	JWT generation/verification and password hashing
Front end	HTML5, CSS3, JavaScript (ES6)	Pages, styling, Fetch API calls to the back end
Interactive prototype	React 19, TypeScript, Tailwind CSS, Vite	Role-based passenger, driver and admin dashboards
Tools	VS Code, Node.js, Git	Development environment

7.1 Description of the Technologies

Python and FastAPI. Python is a simple, readable and widely used language, which makes the back-end code easy to understand and maintain. FastAPI is a modern, high-performance framework built on Python type hints. Request bodies are validated automatically by Pydantic models, and the framework generates interactive documentation at `/docs`, which was used to test every route of AutoCab.

SQLAlchemy and SQLite. SQLite is a light, file-based relational database that needs no separate server. SQLAlchemy is an ORM, so tables are handled as Python classes instead of raw SQL, and the same code can later be moved to PostgreSQL or MySQL by changing only the connection URL.

JWT and bcrypt. A JWT is a signed token that proves the identity of the user without keeping session data on the server. bcrypt is a slow, salted, one-way hashing algorithm that protects passwords even if the database file is leaked.

HTML, CSS and JavaScript. The front end uses plain web technologies with the Fetch API, so it can be hosted on any static web server and opened on any device that has a browser. CSS variables and a responsive grid make the pages adapt to different screen sizes.

React and TypeScript (prototype). The React prototype demonstrates the complete passenger, driver and admin dashboards. TypeScript interfaces describe the same entities as the database tables, which reduces mistakes in the dashboards.

7.2 Reasons for Choosing this Technology Stack

- **Low cost:** every tool is open source, so there is no licence fee for development or deployment.
- **Easy to learn and maintain:** Python and JavaScript have large communities, documentation and examples.
- **Fast development:** FastAPI and SQLAlchemy remove a lot of repeated code for validation, documentation and database access.
- **Light deployment:** the application runs on an ordinary computer (Intel i3, 4 GB RAM) and the entire database is one file that is easy to back up.
- **Scalable design:** because routes, schemas, business logic and models are separate, the project can grow to a bigger database or a mobile front end without rewriting the whole system.

8. SYSTEM CONFIGURATION

8.1 Hardware Requirements

Table 4: Minimum hardware requirements

Component	Minimum requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB
Storage	512 MB free space (source code + database)	2 GB free space
Display	1024 x 768 resolution	1366 x 768 or higher
Network	Local network / localhost	Broadband internet for deployment

8.2 Software Requirements

Table 5: Software requirements

Software	Version / Details
Operating system	Windows 10/11, Linux or macOS
Python	3.10 or higher
Python packages	fastapi, uvicorn, sqlalchemy, pydantic, python-jose, passlib[bcrypt], python-multipart (listed in requirements.txt)
Database	SQLite (bundled with Python)
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest)
Code editor	Visual Studio Code
Node.js (optional)	Required only for the React / Vite dashboard prototype

8.3 Python Dependencies (requirements.txt)

Table 6: Contents of requirements.txt

Package	Minimum version	Use
fastapi	0.104.0	API framework
uvicorn[standard]	0.23.2	ASGI server
sqlalchemy	2.0.22	ORM and database engine
pydantic	2.4.2	Data validation
pydantic-settings	2.0.3	Configuration support
python-jose[cryptography]	3.3.0	JWT encode / decode
passlib[bcrypt]	1.7.4	Password hashing
python-multipart	0.0.6	Form data support for OAuth2

8.4 Installation and Execution Steps

Back end (FastAPI + SQLite)

1. Open a terminal and move to the `backend` folder of the project.
2. Create a virtual environment: `python -m venv venv` and activate it (on Windows: `venv\Scripts\activate`).
3. Install the dependencies using `pip install -r requirements.txt`.
4. Start the server using `uvicorn main:app --reload --host 0.0.0.0 --port 8000`. The tables are created automatically in the file `autocab.db` at first start.
5. Open `http://localhost:8000/docs` to see the Swagger UI where every API can be tested.

Front end (HTML / CSS / JavaScript)

1. Move to the `frontend` folder.
2. Start any static server, for example `python -m http.server 3000`, or use the Live Server extension of VS Code.
3. Open `http://localhost:3000` in the browser. The file `script.js` contains the constant `API_BASE_URL = http://localhost:8000/api` which connects the page to the back end.

8.5 Default Test Accounts

Table 7: Accounts provided for testing

Role	E-mail	Password
Admin	admin@autocab.com	admin123
Driver	john.driver@autocab.com	driver123
Passenger	user@autocab.com	user123

8.6 Project Folder Structure

Table 8: Folder structure of the project

Path	Description
backend/main.py	FastAPI application, CORS configuration and all API routes
backend/database.py	SQLAlchemy engine, session factory and get_db dependency
backend/models.py	ORM models: User, Driver, Cab, Booking
backend/schemas.py	Pydantic schemas for request and response validation
backend/crud.py	Database operations and booking business logic
backend/auth.py	Password hashing, JWT creation, current-user and admin dependencies
frontend/index.html	Home page with vehicle categories
frontend/login.html, register.html	Login and registration forms
frontend/booking.html	Cab list, filter and booking form
frontend/dashboard.html	Booking history of the logged-in user
frontend/script.js	API client: register, login, logout, load cabs, book cab
frontend/style.css	Responsive styling of all pages
src/ (React prototype)	Role-based dashboards written in TypeScript

9. BLOCK DIAGRAM

The block diagram shows the major building blocks of the AutoCab system and the direction of communication between them. The system follows a three-layer architecture: the Presentation Layer, the Application Layer and the Data Layer.

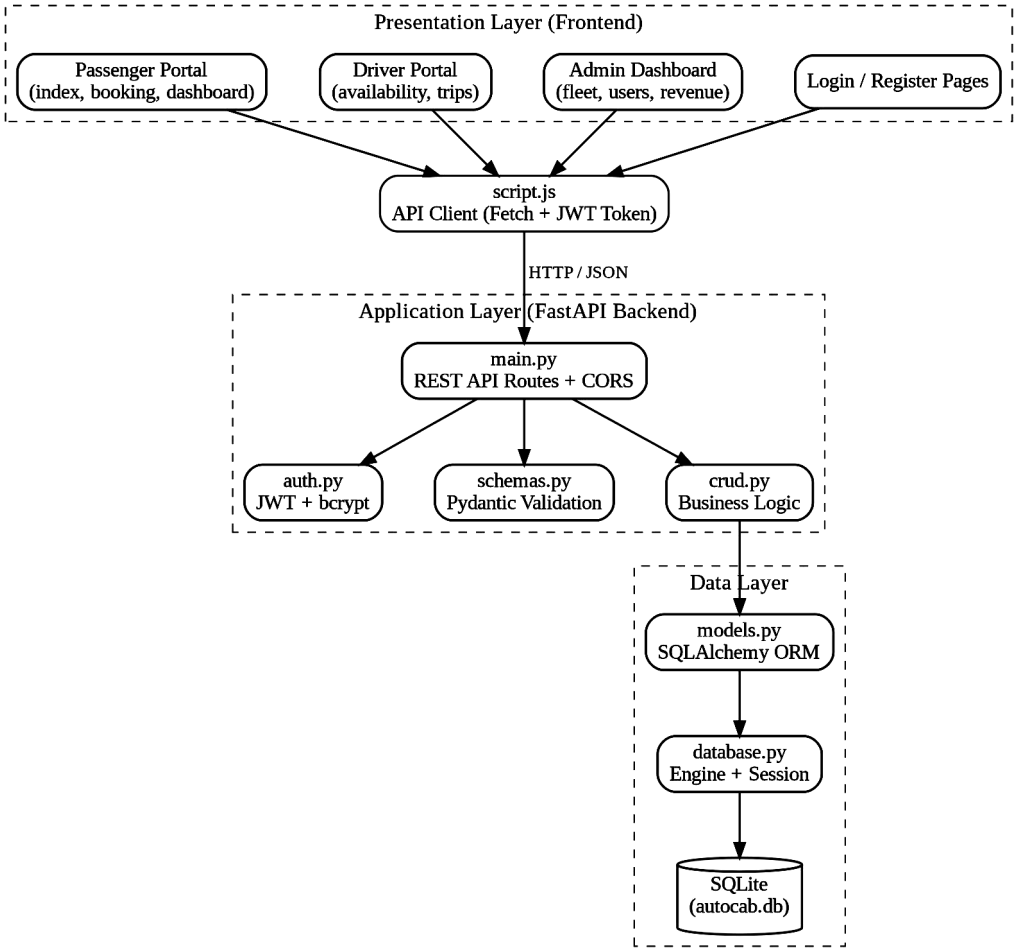


Figure 1: Block diagram of the AutoCab Booking System

9.1 Explanation of the Block Diagram

Presentation layer. This layer contains everything that the user sees in the browser: the passenger pages (home, booking, dashboard), the driver portal, the admin dashboard and the login and registration pages. These pages do not talk to the database directly. They call the JavaScript API client `script.js`.

API client. `script.js` uses the Fetch API to send HTTP requests in JSON format to the back end. It reads the JWT token from localStorage and adds it to the Authorization header of protected requests. The responses are converted into HTML cards and table rows on the page.

Application layer. `main.py` receives the HTTP requests and routes them to the correct function. Incoming data is validated by the Pydantic classes in `schemas.py`. Authentication and authorisation are handled by `auth.py`. The business logic, for example "mark the cab On Ride when a booking is created", is written in `crud.py`.

Data layer. `models.py` describes the tables as Python classes. `database.py` creates the engine and the sessions. The data is finally stored in the SQLite file `autocab.db`. Because the layers are separated, any layer can be changed without rewriting the others; for example, SQLite can be replaced with PostgreSQL by changing only the database URL.

10. DATA FLOW DIAGRAMS (DFD)

A Data Flow Diagram shows how data moves through the system, which processes transform the data, where the data is stored and which external entities send or receive the data. The AutoCab system is explained here with a Level 0 (context) diagram and a Level 1 diagram.

10.1 Level 0 DFD (Context Diagram)

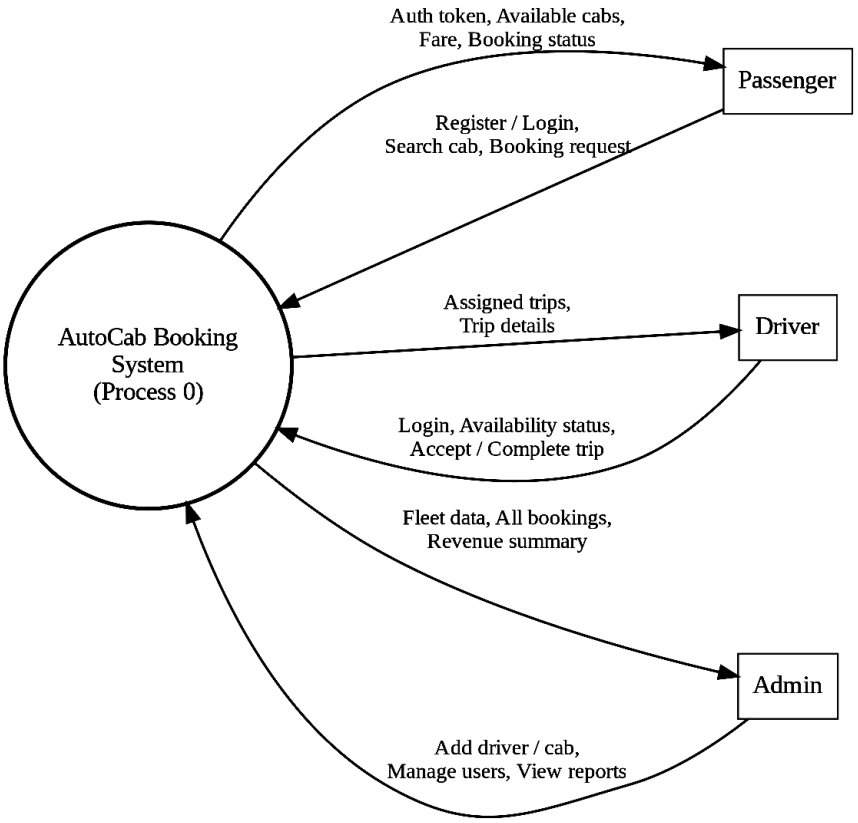


Figure 2: Level 0 DFD of the AutoCab Booking System

At Level 0 the entire system is represented by a single process. There are three external entities. The **Passenger** sends registration or login details, cab search requests and booking requests, and receives the authentication token, the list of available cabs, the fare and the booking status. The **Driver** sends login details, availability status and trip actions (accept or complete), and receives the assigned trips. The **Admin** sends commands to add drivers and cabs and to manage users, and receives fleet data, all bookings and revenue summaries.

10.2 Level 1 DFD

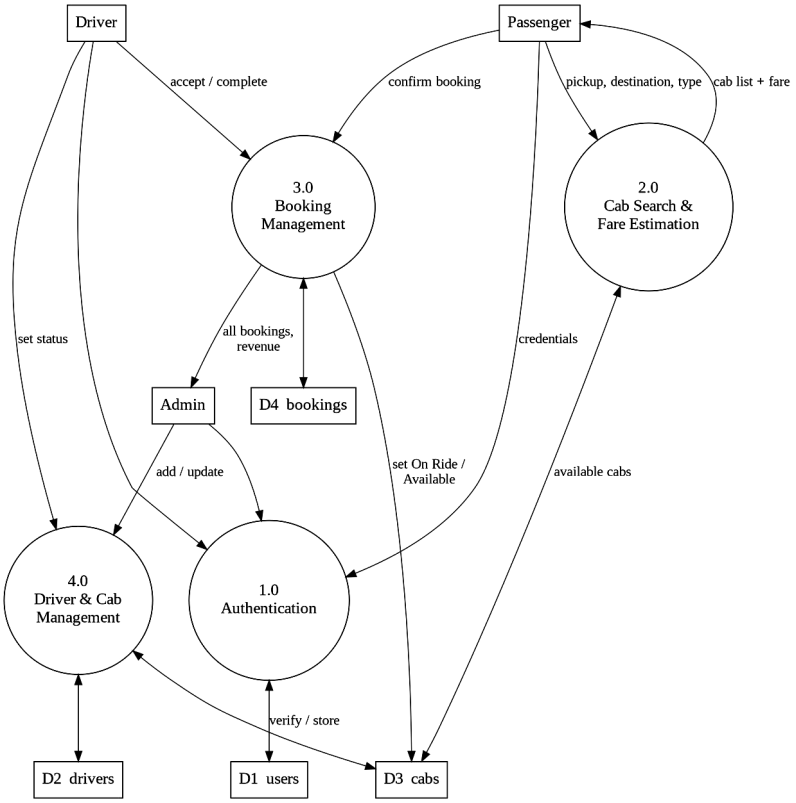


Figure 3: Level 1 DFD showing the four main processes and data stores

At Level 1 the system is broken into four processes and four data stores.

- **Process 1.0 Authentication** receives credentials from all three entities, checks them against the users store (D1) and returns a JWT token. New registrations are written into D1 after the password is hashed.
- **Process 2.0 Cab Search and Fare Estimation** receives the pickup, destination and vehicle type from the passenger, reads the available cabs from the cabs store (D3) and returns the cab list together with the estimated fare.
- **Process 3.0 Booking Management** receives the booking confirmation from the passenger and the accept / complete actions from the driver. It writes to the bookings store (D4) and updates the cabs store (D3) to "On Ride" or "Available". It also supplies all bookings and revenue information to the admin.
- **Process 4.0 Driver and Cab Management** allows the admin to add or update drivers (D2) and cabs (D3) and allows drivers to change their own status.

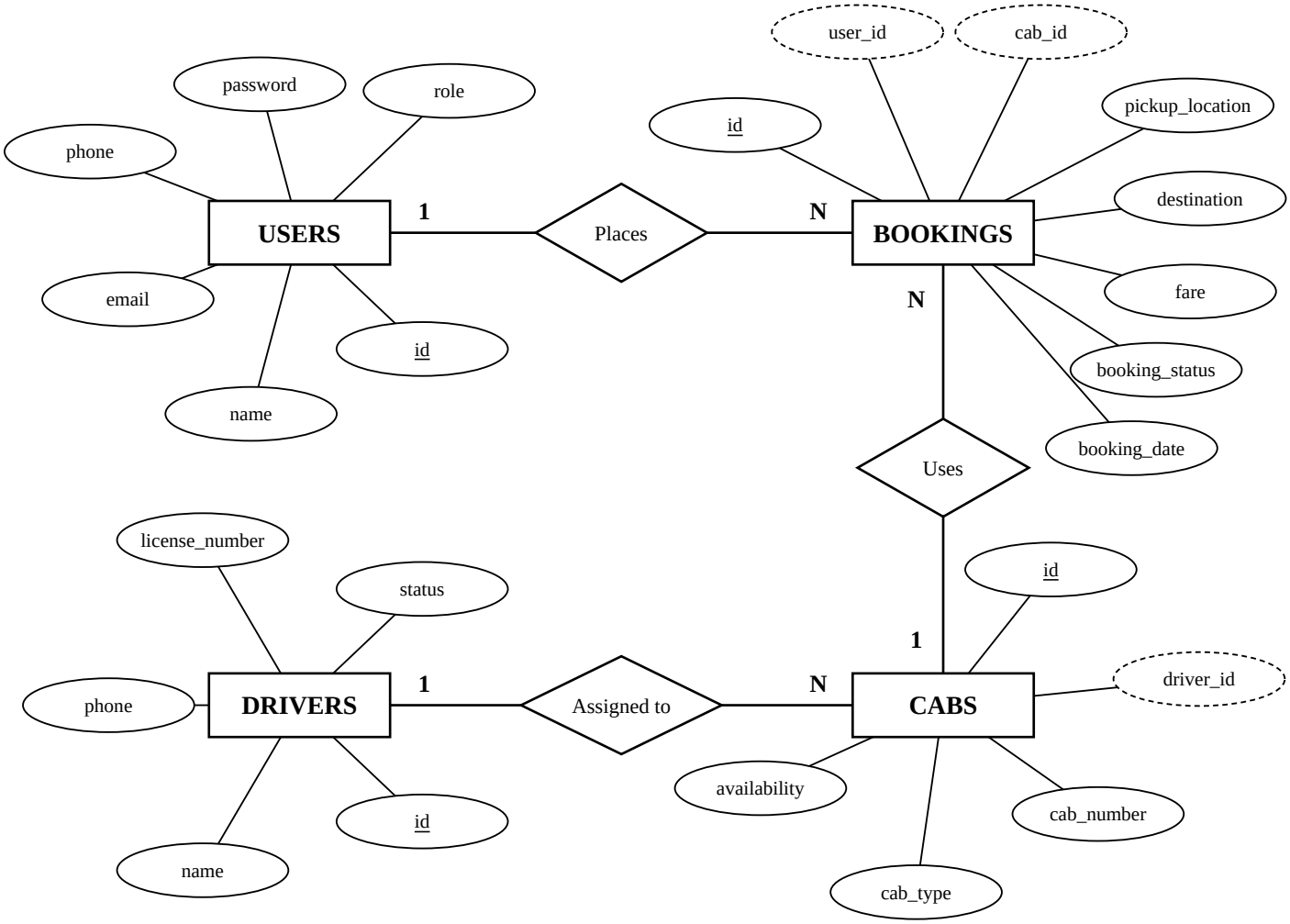
10.3 Data Stores

Table 9: Data stores used in the DFD

Store	Table	Data held
D1	users	Name, e-mail, phone, hashed password, role
D2	drivers	Name, phone, licence number, status
D3	cabs	Cab number, category, availability, driver link
D4	bookings	User, cab, pickup, destination, fare, status, date

11. ENTITY RELATIONSHIP (ER) DIAGRAM

The Entity Relationship diagram shows the tables of the database, their attributes and the relationships between them. AutoCab uses four entities: Users, Drivers, Cabs and Bookings. The diagram has been prepared from the SQLAlchemy models defined in `models.py`.



Legend: underlined = primary key, dashed oval = foreign key, 1 / N = cardinality

Figure 4: ER diagram of the AutoCab database

11.1 Relationships

- **Users – Bookings (1 : N):** one user can place many bookings, but each booking belongs to exactly one user (`bookings.user_id` references `users.id`).
- **Cabs – Bookings (1 : N):** one cab can be used in many bookings over time, but each booking uses exactly one cab (`bookings.cab_id` references `cabs.id`).
- **Drivers – Cabs (1 : N):** one driver can be linked to cabs, and each cab has at most one driver (`cabs.driver_id` references `drivers.id`, nullable).

11.2 Table Structures

Table 10: Table: users

Column	Data type	Constraint	Description
id	Integer	Primary key, indexed	Unique user number
name	String	Not null	Full name
email	String	Unique, indexed, not null	Login ID
phone	String	Not null	Contact number
password	String	Not null	bcrypt hash of the password
role	String	Default "user"	user / driver / admin

Table 11: Table: drivers

Column	Data type	Constraint	Description
id	Integer	Primary key, indexed	Unique driver number
name	String	Not null	Driver name
phone	String	Not null	Contact number
license_number	String	Unique, not null	Driving licence number
status	String	Default "Available"	Available / Busy / Offline

Table 12: Table: cabs

Column	Data type	Constraint	Description
id	Integer	Primary key, indexed	Unique cab number
driver_id	Integer	Foreign key (drivers.id), nullable	Assigned driver
cab_number	String	Unique, not null	Registration plate number
cab_type	String	Not null	Sedan / SUV / Luxury / EV
availability	String	Default "Available"	Available / On Ride / Maintenance

Table 13: Table: bookings

Column	Data type	Constraint	Description
id	Integer	Primary key, indexed	Unique booking number
user_id	Integer	Foreign key (users.id), not null	Passenger who booked
cab_id	Integer	Foreign key (cabs.id), not null	Cab used for the trip
pickup_location	String	Not null	Starting point
destination	String	Not null	Drop point
fare	Float	Not null	Fare in rupees
booking_status	String	Default "Pending"	Pending / Accepted / In Progress / Completed / Cancelled
booking_date	DateTime	Default current UTC time	Date and time of booking

11.3 Use Case Diagram

The use case diagram summarises the functions available to each type of actor. All actors share the Register / Login use case; the other functions are specific to the role.

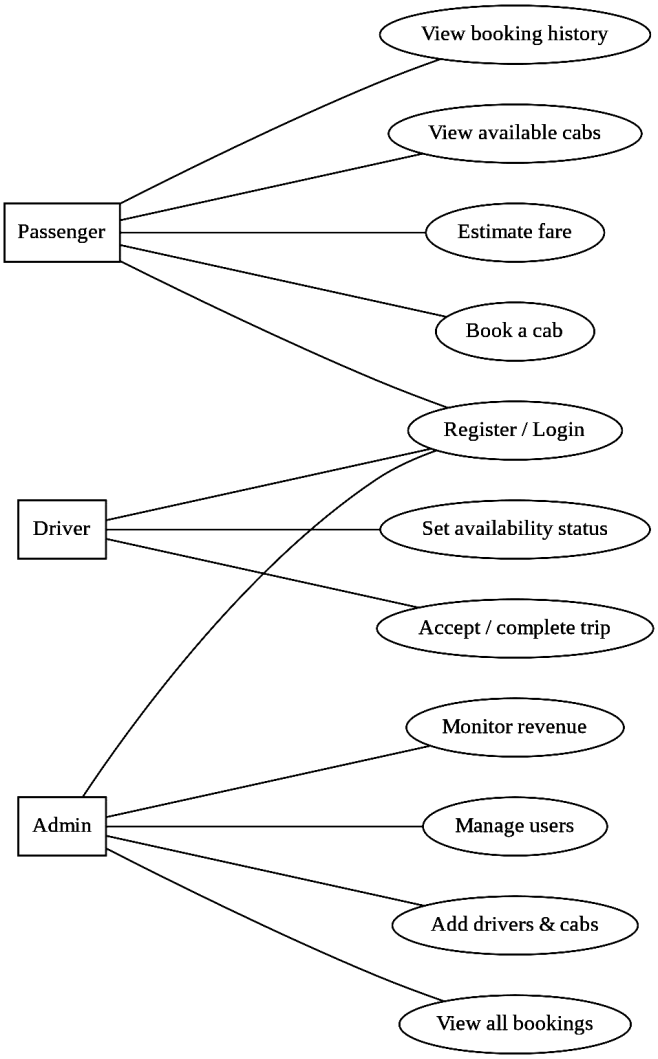


Figure 5: Use case diagram of the AutoCab system

12. OUTPUT SCREENS

This chapter presents the screens of the AutoCab front end. The screenshots were taken from the HTML pages of the project running in a browser at 1280 x 760 resolution. The sample data shown on the screens (cab numbers, booking IDs, fares) is the demonstration data of the project.

12.1 Home Page

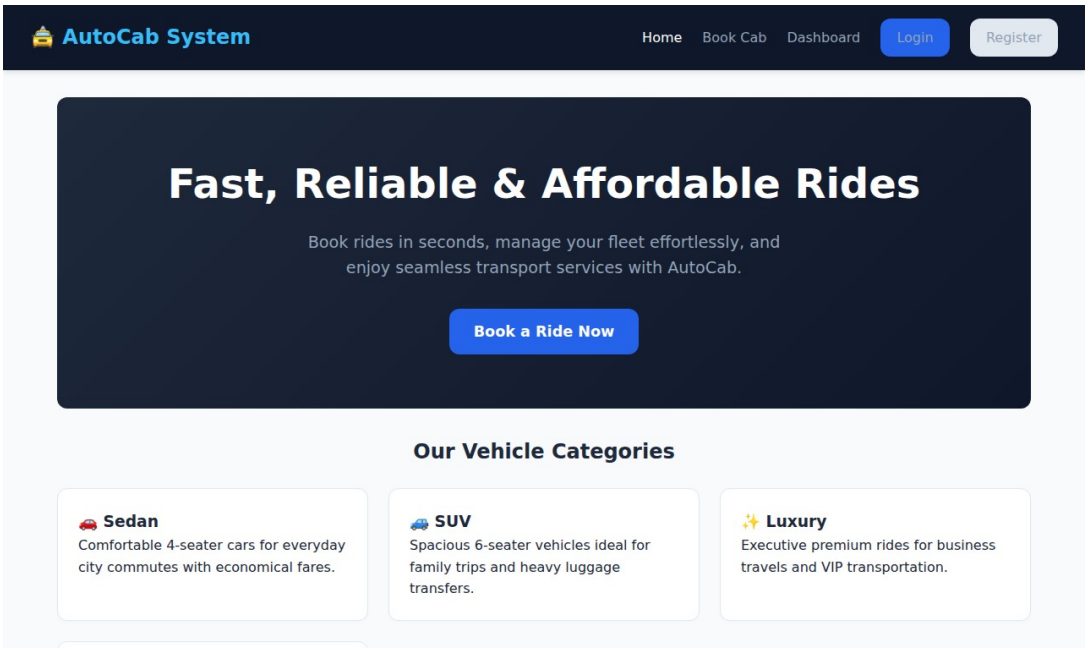


Figure 6: Home page of AutoCab

The home page is the entry point of the application. The dark navigation bar at the top has the AutoCab logo and links for Home, Book Cab, Dashboard, Login and Register. Below it the hero section carries the headline "Fast, Reliable & Affordable Rides" with the button **Book a Ride Now**. The section "Our Vehicle Categories" displays four cards: Sedan (comfortable 4-seater), SUV (spacious 6-seater), Luxury (executive rides) and Electric EV (eco-friendly vehicles). This page is public; no login is needed to view it.

12.2 Registration Page

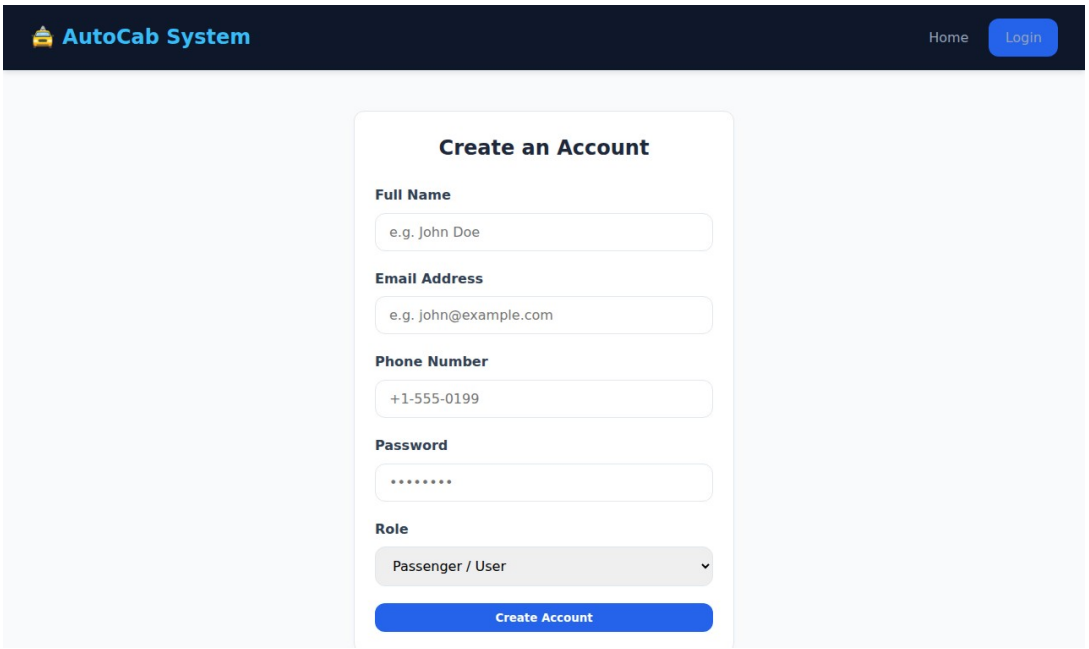


Figure 7: User registration page

The registration page contains the fields Full Name, Email Address, Phone Number, Password and Role. The role list offers Passenger / User, Cab Driver and Fleet Admin. On clicking **Create Account** the function `handleRegister()` sends the data to `POST /api/auth/register`. If the e-mail already exists, the server answers with the message "Email address is already registered." and the page shows it in an alert; otherwise the user is redirected to the login page.

12.3 Login Page

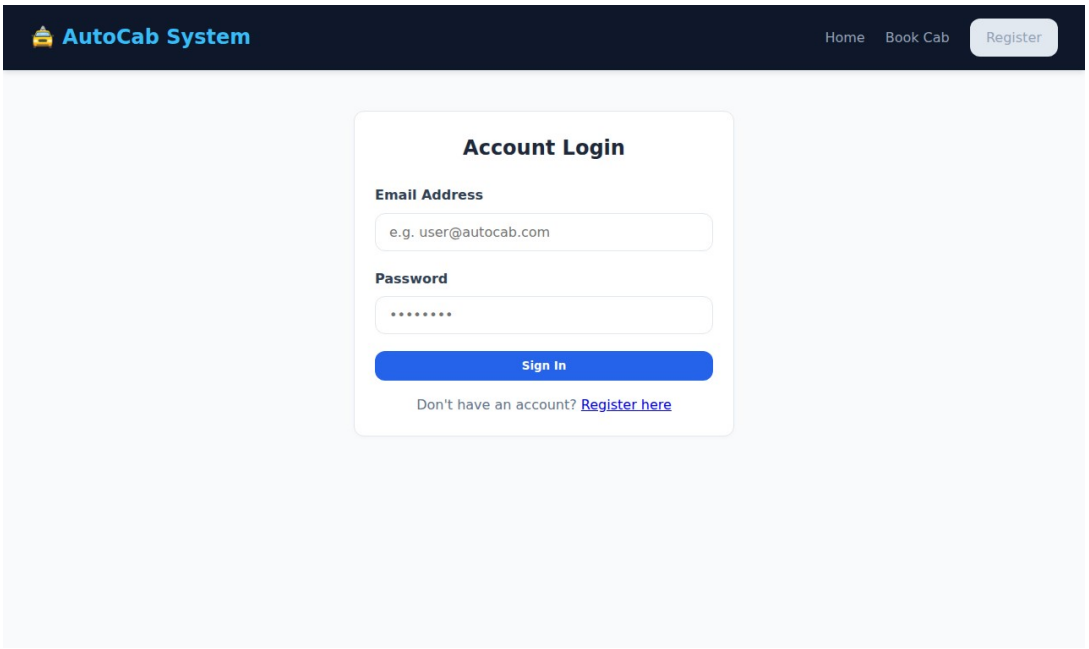


Figure 8: Account login page

The login page asks for an e-mail address and password. The function `handleLogin()` calls `POST /api/auth/login`. On success, the JWT token and the user object are stored in the browser localStorage and the user is taken to the dashboard. When the e-mail or the password is wrong the server returns HTTP 401 with the message "Incorrect email or password".

12.4 Booking Page

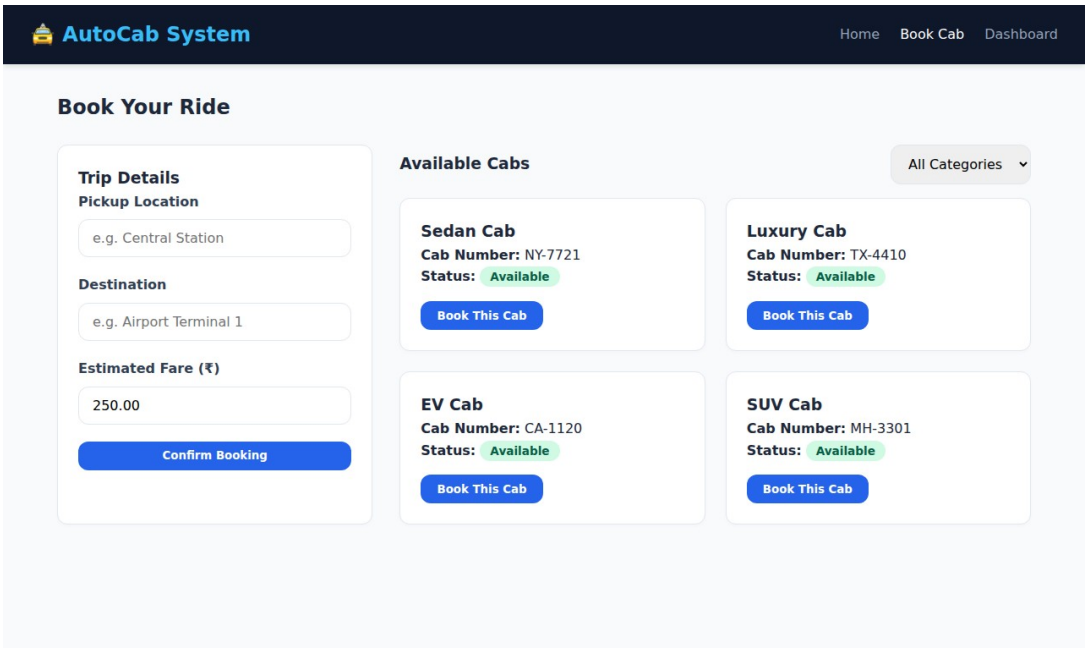


Figure 9: Booking page with available cabs

The booking page is divided into two parts. The left card, **Trip Details**, contains the fields Pickup Location, Destination and Estimated Fare (in rupees, read-only) with the button **Confirm Booking**. The right part, **Available Cabs**, lists every cab whose availability is "Available" as a card showing the category, the cab number and a green status badge. The drop-down at the top right filters the list by All Categories, Sedan, SUV, Luxury or EV; every change calls `loadAvailableCabs()` again. Pressing **Book This Cab** stores the selected cab ID, and **Confirm Booking** submits the booking to `POST /api/bookings` with the JWT token. If the user is not logged in, the page asks the user to log in first.

12.5 User Dashboard

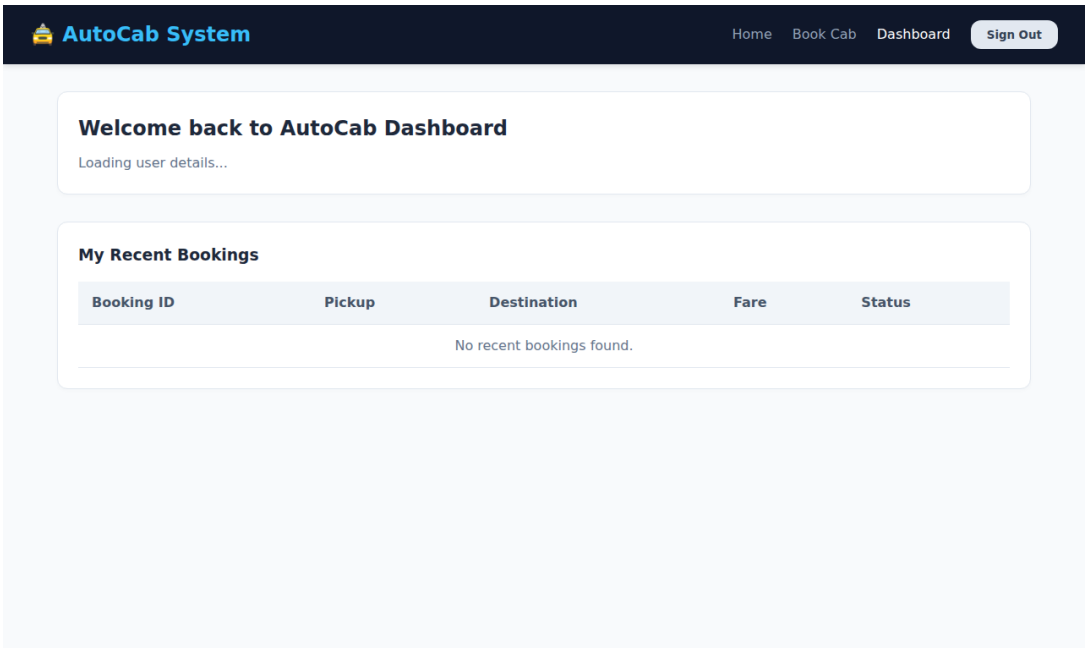


Figure 10: User dashboard with recent bookings

After login the user reaches the dashboard. The welcome card shows the role of the user and the table **My Recent Bookings** lists the Booking ID, Pickup, Destination, Fare and Status of every booking, newest first. The data comes from `GET /api/bookings/my`. The **Sign Out** button removes the token and user details from localStorage and returns to the login page.

12.6 Role-Based Dashboards of the React Prototype

In addition to the plain HTML pages, the project contains a React and TypeScript prototype (folder `src/components`) which implements the role-based dashboards of the synopsis. The **User Dashboard** shows active rides and the trip history with a receipt view. The **Driver Portal** has status buttons (Available, Busy, Offline), earnings and completed ride counters, incoming ride requests with an Accept button, and Start Trip / Complete Trip buttons for the active ride. The **Fleet Admin Portal** has five tabs: Overview (statistics and quick actions), Users (with delete), Drivers (with the Register New Driver form), Cabs (with the Add New Fleet Cab form and the options to mark a cab Available or Maintenance) and Bookings (global booking log with status update).

13. PROJECT EXPLANATION

13.1 Overall Working of the System

When the application is started, `main.py` first calls `models.Base.metadata.create_all()`, which creates the four tables in `autocab.db` if they do not already exist. The FastAPI application is then configured with the CORS middleware so that the front end, which runs on a different port, is allowed to call the API. Each request passes through the following path: the browser sends a JSON request, FastAPI matches the URL to a route function, Pydantic validates the body, the dependency functions in `auth.py` check the token and role, the function in `crud.py` reads or writes the database through SQLAlchemy, and the result is converted by a response schema into JSON and returned to the browser.

13.2 Request Handling Flow for Login

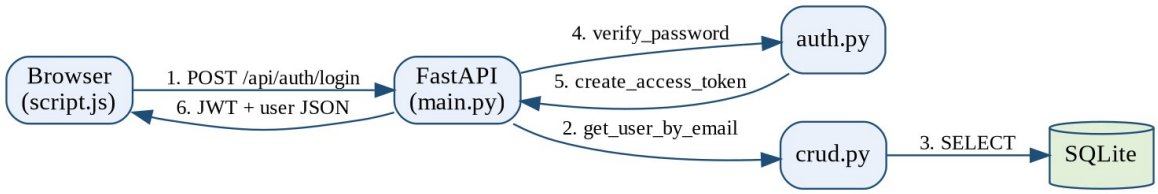


Figure 11: Message flow of the login request

The login request proves how the layers work together. (1) The browser sends the e-mail and the password to `/api/auth/login`. (2) The route calls `crud.get_user_by_email()`. (3) SQLAlchemy runs a SELECT on the users table. (4) `auth.verify_password()` compares the entered password with the stored bcrypt hash. (5) If the password matches, `auth.create_access_token()` creates the JWT. (6) The route returns the token and the user details; the password hash is never returned because the response schema `UserResponse` does not contain it.

13.3 Booking Process Flow

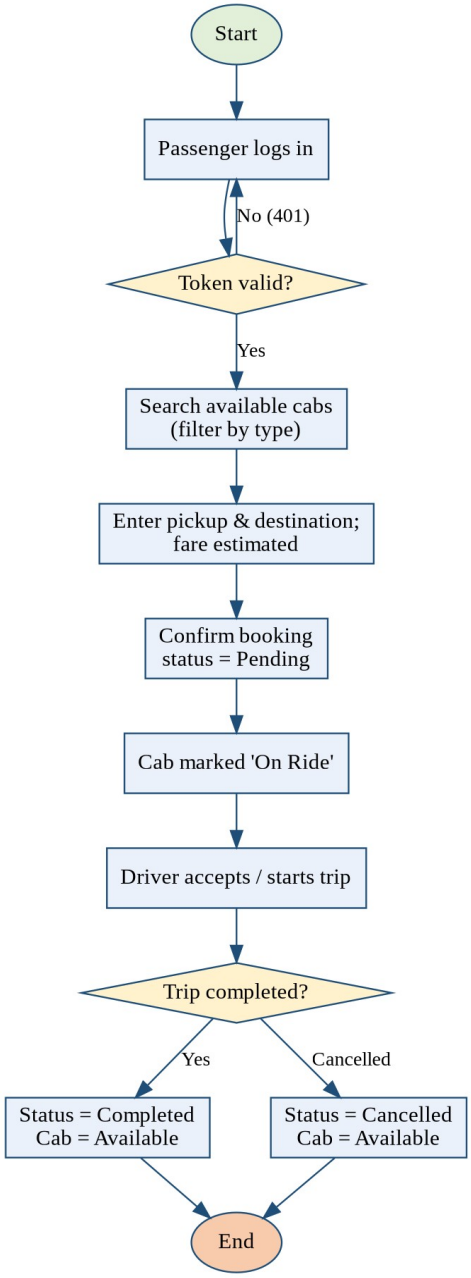


Figure 12: Flowchart of the cab booking process

The flowchart shows the complete booking process. A passenger must be logged in, because the booking route depends on `get_current_user`. After the passenger selects a cab and fills in the trip details, the booking is saved with the status Pending and the cab is marked On Ride. The trip then proceeds until it is completed or cancelled, and in both cases the cab is released.

13.4 Fare Estimation Logic

The fare is calculated from the distance of the trip and the per-kilometre rate of the vehicle category. In the React booking page the formula is: **Fare = (Distance in km × Rate per km) + Base fare of ₹30**. The sample rates used in the project data are shown below. For example, a Sedan trip of 18.2 km costs 18.2 × 18 + 30 = ₹357.60, which is exactly the fare stored in the sample booking 101. In the plain HTML booking page the estimated fare field is read-only and has a default value of ₹250.00, which is submitted with the booking. The calculated fare is sent to the API and stored in the `fare` column so that it cannot change later.

Table 14: Sample rate per kilometre by category

Category	Rate (₹ per km)	Typical use
Sedan	18.00	Everyday city travel
SUV	25.00	Family trips and luggage
Luxury	45.00	Business and VIP travel
EV	20.00	Eco-friendly travel

13.5 Booking Status Life Cycle

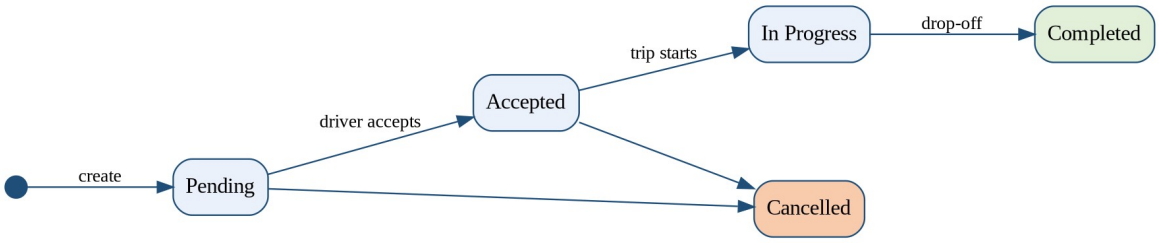


Figure 13: Booking status transitions

A booking starts as **Pending** when the passenger confirms it. The driver can **Accept** it, after which the driver starts the trip and the status becomes **In Progress**. When the passenger is dropped, the status is changed to **Completed** and revenue is counted. A booking can be **Cancelled** while it is Pending or Accepted. The function `update_booking_status()` contains the rule that makes the system consistent: if the new status is Completed or Cancelled, the related cab is set back to Available.

13.6 Cab Availability Synchronisation

Table 15: Effect of booking events on the cab and driver

Event	**Booking status**	**Cab availability**	**Driver status (React prototype)**
Booking created	Pending	Available → On Ride	Available → Busy
Driver accepts	Accepted	On Ride	Busy
Trip starts	In Progress	On Ride	Busy
Trip completed	Completed	On Ride → Available	Busy → Available (rides +1)
Booking cancelled	Cancelled	On Ride → Available	Busy → Available

13.7 Description of the API Endpoints

Table 16: REST API endpoints of the AutoCab back end

Method	URL	Access	Function
GET	/	Public	Health message of the API
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Login and obtain JWT token
GET	/api/auth/me	Logged in	Profile of the current user
GET	/api/users	Admin	List all users
DELETE	/api/users/{id}	Admin	Delete a user
GET	/api/drivers	Public	List drivers
POST	/api/drivers	Admin	Register a driver
PUT	/api/drivers/{id}/status	Public	Change driver status
GET	/api/cabs	Public	List all cabs
GET	/api/cabs/available	Public	List available cabs, optional cab_type filter
POST	/api/cabs	Admin	Add a cab
POST	/api/bookings	Logged in	Create a booking
GET	/api/bookings/my	Logged in	Bookings of the current user
GET	/api/bookings/all	Admin	All bookings
PUT	/api/bookings/{id}/status	Public	Update booking status

13.8 Security Design

- **Password protection:** passwords are hashed with bcrypt through passlib; only the hash is stored in the database.
- **Token based authentication:** the JWT is signed with the HS256 algorithm and a secret key, carries an expiry time of 24 hours and is checked on every protected route.
- **Role based authorisation:** the dependency `require_admin` blocks non-admin users from the admin routes with HTTP 403.
- **Input validation:** Pydantic schemas reject wrong data types and invalid e-mail addresses before the data reaches the database.
- **SQL injection protection:** SQLAlchemy uses parameterised queries, so user input is never joined into SQL strings.
- **Cross-origin control:** the CORS middleware decides which front-end origins may call the API.

13.9 Front-End Explanation

The front end is intentionally simple. Each HTML page loads `style.css` for the design and `script.js` for behaviour. The CSS file defines colour variables (primary blue, dark navy, success green, danger red), a responsive grid for cards, styled tables, form controls and badges. `script.js` contains small, single-purpose functions: `getToken()` and `getUser()` read the login data, `handleRegister()` and `handleLogin()` call the authentication API, `logout()` clears the session, `loadAvailableCabs()` builds the cab cards from the API response, and `handleBooking()` sends the booking request after checking that the user is logged in.

13.10 Role of the React / TypeScript Prototype

To demonstrate the complete three-portal design described in the synopsis, a React prototype has also been developed. The file `types.ts` defines the TypeScript interfaces for User, Driver, Cab, Booking and SystemStats; `App.tsx` keeps the application state and switches between the Home, Booking, User Dashboard, Driver Dashboard and Admin Dashboard screens according to the role of the logged-in user; the components `BookingPage.tsx`, `UserDashboard.tsx`, `DriverDashboard.tsx` and `AdminDashboard.tsx` implement each screen. The prototype uses the same entities, statuses and fare logic as the FastAPI back end, so the two parts describe the same data model.

14. CODING (SAMPLE WITH EXPLANATION)

This chapter contains the source code of the AutoCab system together with an explanation of each file. The back-end files are described first, in the order in which a request travels through them (database, models, schemas, authentication, CRUD operations and finally the routes), followed by the front-end files and the key parts of the React prototype. Each listing is the actual code of the project.

14.1 Back End: database.py

This file creates the connection between the application and the SQLite database.

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# SQLite Database URL
SQLALCHEMY_DATABASE_URL = "sqlite:///./autocab.db"

# For SQLite, check_same_thread=False is needed for multi-threaded FastAPI requests
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# Dependency to get DB session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Explanation

- `SQLALCHEMY_DATABASE_URL = "sqlite:///./autocab.db"` tells SQLAlchemy to use a SQLite file named `autocab.db` in the current folder. To move to PostgreSQL or MySQL only this URL needs to be changed.
- `check_same_thread=False` is needed because FastAPI may use different threads for different requests.
- `SessionLocal` creates a session for every request; `autocommit=False` means changes are saved only on `commit()`.
- `Base = declarative_base()` is the parent of all ORM models. `get_db()` opens a session, gives it to the route with `yield` and always closes it in `finally`.

14.2 Back End: models.py

The models describe the four tables of the database as Python classes.

```
from sqlalchemy import Column, Integer, String, Float, ForeignKey, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    phone = Column(String, nullable=False)
    password = Column(String, nullable=False)
    role = Column(String, default="user")  # 'user', 'driver', 'admin'

    bookings = relationship("Booking", back_populates="user")


class Driver(Base):
    __tablename__ = "drivers"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    phone = Column(String, nullable=False)
    license_number = Column(String, unique=True, nullable=False)
    status = Column(String, default="Available")  # 'Available', 'Busy', 'Offline'

    cabs = relationship("Cab", back_populates="driver")


class Cab(Base):
    __tablename__ = "cabs"

    id = Column(Integer, primary_key=True, index=True)
    driver_id = Column(Integer, ForeignKey("drivers.id"), nullable=True)
    cab_number = Column(String, unique=True, nullable=False)
    cab_type = Column(String, nullable=False)  # 'Sedan', 'SUV', 'Luxury', 'EV'
    availability = Column(String, default="Available")  # 'Available', 'On Ride',
'Maintenance'

    driver = relationship("Driver", back_populates="cabs")
    bookings = relationship("Booking", back_populates="cab")


class Booking(Base):
    __tablename__ = "bookings"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    cab_id = Column(Integer, ForeignKey("cabs.id"), nullable=False)
    pickup_location = Column(String, nullable=False)
    destination = Column(String, nullable=False)
    fare = Column(Float, nullable=False)
    booking_status = Column(String, default="Pending")  # 'Pending', 'Accepted', 'In
Progress', 'Completed', 'Cancelled'
    booking_date = Column(DateTime, default=datetime.utcnow)

    user = relationship("User", back_populates="bookings")
    cab = relationship("Cab", back_populates="bookings")
```

Explanation

- Every class inherits from `Base` and its `__tablename__` gives the name of the table.
- `Column(Integer, primary_key=True, index=True)` creates an auto-incrementing primary key with an index for fast search.

- `unique=True` on `User.email`, `Driver.license_number` and `Cab.cab_number` prevents duplicate records at database level.
- `ForeignKey("drivers.id")` in `Cab`, and `ForeignKey("users.id")` and `ForeignKey("cabs.id")` in `Booking`, implement the relationships of the ER diagram.
- `relationship()` with `back_populates` lets the code move between related objects, for example `booking.user` or `driver.cabs`, without writing JOIN queries.
- `booking_date` uses `default=datetime.utcnow`, so the time of booking is stored automatically.

14.3 Back End: schemas.py

Schemas are Pydantic classes that define the exact shape of the data that enters and leaves the API.

```
from pydantic import BaseModel, EmailStr
from typing import Optional, List
from datetime import datetime

# User Schemas
class UserBase(BaseModel):
    name: str
    email: EmailStr
    phone: str
    role: Optional[str] = "user"

class UserCreate(UserBase):
    password: str

class UserResponse(UserBase):
    id: int

    class Config:
        from_attributes = True

# Auth Schemas
class Token(BaseModel):
    access_token: str
    token_type: str
    user: UserResponse

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

# Driver Schemas
class DriverBase(BaseModel):
    name: str
    phone: str
    license_number: str
    status: Optional[str] = "Available"

class DriverCreate(DriverBase):
    pass

class DriverResponse(DriverBase):
    id: int

    class Config:
        from_attributes = True

# Cab Schemas
class CabBase(BaseModel):
    driver_id: Optional[int] = None
    cab_number: str
    cab_type: str
    availability: Optional[str] = "Available"

class CabCreate(CabBase):
    pass

class CabResponse(CabBase):
    id: int
    driver: Optional[DriverResponse] = None

    class Config:
        from_attributes = True

# Booking Schemas
class BookingBase(BaseModel):
    cab_id: int
    pickup_location: str
    destination: str
    fare: float

class BookingCreate(BookingBase):
    user_id: Optional[int] = None
```

```
class BookingStatusUpdate(BaseModel):
    booking_status: str

class BookingResponse(BookingBase):
    id: int
    user_id: int
    booking_status: str
    booking_date: datetime
    user: Optional[UserResponse] = None
    cab: Optional[CabResponse] = None

    class Config:
        from_attributes = True
```

Explanation

- The pattern Base / Create / Response is used for every entity. The Base class holds common fields, the Create class adds fields needed only at creation (for example `password`), and the Response class adds the database `id` and related objects.
- `EmailStr` makes Pydantic reject an invalid e-mail address automatically.
- `UserResponse` does not contain the password field, therefore the hash can never leak in an API response.
- `from_attributes = True` (the Config class) lets Pydantic read data directly from SQLAlchemy objects.
- `BookingResponse` includes nested `user` and `cab` objects, so one API call returns complete booking details.
- `BookingStatusUpdate` carries only the new `booking_status` value for the status update route.

14.4 Back End: auth.py

This file implements password hashing, token creation and the security dependencies.

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
import models, database

SECRET_KEY = "autocab-super-secret-jwt-key-for-development"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def get_current_user(token: str = Depends(oauth2_scheme), db: Session =
Depends(database.get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email: str = payload.get("sub")
        if email is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception


    user = db.query(models.User).filter(models.User.email == email).first()
    if user is None:
        raise credentials_exception
    return user

def require_admin(current_user: models.User = Depends(get_current_user)):
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Operation restricted to admin users only."
        )
    return current_user
```

Explanation

- `CryptContext(schemes=["bcrypt"])` is the password hashing engine; `get_password_hash()` hashes a password and `verify_password()` compares a plain password with a stored hash.

- `create_access_token()` copies the data (the user e-mail as "sub"), adds an expiry time (`exp`) of 24 hours (`60 * 24` minutes) and signs it with the secret key using HS256.
- `OAuth2PasswordBearer` tells FastAPI to read the token from the `Authorization: Bearer` header and also shows the Authorize button in Swagger UI.
- `get_current_user()` decodes the token, extracts the e-mail, loads the user from the database and raises HTTP 401 for any problem (missing token, bad signature, expired token, unknown user).
- `require_admin()` builds on `get_current_user()` and raises HTTP 403 if the role is not "admin". It is attached to routes that only the fleet owner may use.
- **Note:** the secret key is written in the source code only for development. In a production deployment it must be read from an environment variable.

14.5 Back End: crud.py

This file contains all database operations and the business rules of the application.

```
from sqlalchemy.orm import Session
import models, schemas, auth

# User CRUD
def get_user_by_email(db: Session, email: str):
    return db.query(models.User).filter(models.User.email == email).first()

def create_user(db: Session, user: schemas.UserCreate):
    hashed_password = auth.get_password_hash(user.password)
    db_user = models.User(
        name=user.name,
        email=user.email,
        phone=user.phone,
        password=hashed_password,
        role=user.role or "user"
    )
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

def get_users(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.User).offset(skip).limit(limit).all()

def delete_user(db: Session, user_id: int):
    user = db.query(models.User).filter(models.User.id == user_id).first()
    if user:
        db.delete(user)
        db.commit()
        return True
    return False

# Driver CRUD
def get_drivers(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Driver).offset(skip).limit(limit).all()

def create_driver(db: Session, driver: schemas.DriverCreate):
    db_driver = models.Driver(**driver.model_dump())
    db.add(db_driver)
    db.commit()
    db.refresh(db_driver)
    return db_driver

def update_driver_status(db: Session, driver_id: int, status: str):
    driver = db.query(models.Driver).filter(models.Driver.id == driver_id).first()
    if driver:
        driver.status = status
        db.commit()
        db.refresh(driver)
    return driver

# Cab CRUD
def get_cabs(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Cab).offset(skip).limit(limit).all()

def create_cab(db: Session, cab: schemas.CabCreate):
    db_cab = models.Cab(**cab.model_dump())
    db.add(db_cab)
    db.commit()
    db.refresh(db_cab)
    return db_cab

def get_available_cabs(db: Session, cab_type: str = None):
    query = db.query(models.Cab).filter(models.Cab.availability == "Available")
    if cab_type and cab_type != "All":
        query = query.filter(models.Cab.cab_type == cab_type)
    return query.all()

# Booking CRUD
def create_booking(db: Session, booking: schemas.BookingCreate, user_id: int):
    db_booking = models.Booking(
        user_id=user_id,
        cab_id=booking.cab_id,
        pickup_location=booking.pickup_location,
```

```
        destination=booking.destination,
        fare=booking.fare,
        booking_status="Pending"
    )
    db.add(db_booking)

    # Mark Cab as 'On Ride'
    cab = db.query(models.Cab).filter(models.Cab.id == booking.cab_id).first()
    if cab:
        cab.availability = "On Ride"

    db.commit()
    db.refresh(db_booking)
    return db_booking

def get_user_bookings(db: Session, user_id: int):
    return db.query(models.Booking).filter(models.Booking.user_id ==
user_id).order_by(models.Booking.booking_date.desc()).all()

def get_all_bookings(db: Session):
    return db.query(models.Booking).order_by(models.Booking.booking_date.desc()).all()

def update_booking_status(db: Session, booking_id: int, status: str):
    booking = db.query(models.Booking).filter(models.Booking.id == booking_id).first()
    if booking:
        booking.booking_status = status
        if status in ["Completed", "Cancelled"]:
            cab = db.query(models.Cab).filter(models.Cab.id == booking.cab_id).first()
            if cab:
                cab.availability = "Available"
        db.commit()
        db.refresh(booking)
    return booking
```

Explanation

- `create_user()` hashes the password before saving, and sets the role to "user" when no role is given.
- `get_available_cabs()` filters cabs where `availability == "Available"`; if a `cab_type` other than "All" is given, a second filter is applied.
- `create_booking()` is the most important function. It saves the booking with the status "Pending" and in the same transaction changes the cab to "On Ride", so booking and availability always change together.
- `get_user_bookings()` returns only the bookings of one user, newest first (`order_by(booking_date.desc())`).
- `update_booking_status()` changes the status and, when the new status is "Completed" or "Cancelled", releases the cab by setting it back to "Available". This single rule prevents a cab from remaining blocked after a trip.
- Every write operation ends with `commit()` and `refresh()` so that the returned object contains the latest values from the database.

14.6 Back End: main.py

This is the entry point of the application. It creates the tables, configures CORS and defines every API route.

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from typing import List, Optional
import models, schemas, crud, auth, database

# Create tables in SQLite
models.Base.metadata.create_all(bind=database.engine)

app = FastAPI(
    title="AutoCab System API",
    description="Backend RESTful API services for Cab Booking and Fleet Management System",
    version="1.0.0"
)

# CORS Middleware Configuration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def read_root():
    return {"status": "online", "message": "AutoCab System API is up and running"}

# --- AUTHENTICATION ROUTES ---
@app.post("/api/auth/register", response_model=schemas.UserResponse)
def register_user(user: schemas.UserCreate, db: Session = Depends(database.get_db)):
    db_user = crud.get_user_by_email(db, email=user.email)
    if db_user:
        raise HTTPException(status_code=400, detail="Email address is already registered.")
    return crud.create_user(db=db, user=user)

@app.post("/api/auth/login", response_model=schemas.Token)
def login_for_access_token(login_data: schemas.LoginRequest, db: Session =
Depends(database.get_db)):
    user = crud.get_user_by_email(db, email=login_data.email)
    if not user or not auth.verify_password(login_data.password, user.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = auth.create_access_token(data={"sub": user.email})
    return {
        "access_token": access_token,
        "token_type": "bearer",
        "user": user
    }

@app.get("/api/auth/me", response_model=schemas.UserResponse)
def get_current_user_profile(current_user: models.User = Depends(auth.get_current_user)):
    return current_user

# --- USER ROUTES ---
@app.get("/api/users", response_model=List[schemas.UserResponse])
def read_users(db: Session = Depends(database.get_db), current_user: models.User =
Depends(auth.require_admin)):
    return crud.get_users(db)

@app.delete("/api/users/{user_id}")
def delete_user_route(user_id: int, db: Session = Depends(database.get_db), current_user:
models.User = Depends(auth.require_admin)):
    success = crud.delete_user(db, user_id)
    if not success:
        raise HTTPException(status_code=404, detail="User not found")
    return {"message": "User deleted successfully"}

# --- DRIVER ROUTES ---
```

```
@app.get("/api/drivers", response_model=List[schemas.DriverResponse])
def read_drivers(db: Session = Depends(database.get_db)):
    return crud.get_drivers(db)

@app.post("/api/drivers", response_model=schemas.DriverResponse)
def create_new_driver(driver: schemas.DriverCreate, db: Session = Depends(database.get_db),
current_user: models.User = Depends(auth.require_admin)):
    return crud.create_driver(db, driver)

@app.put("/api/drivers/{driver_id}/status", response_model=schemas.DriverResponse)
def update_driver_status_route(driver_id: int, status: str, db: Session =
Depends(database.get_db)):
    updated_driver = crud.update_driver_status(db, driver_id, status)
    if not updated_driver:
        raise HTTPException(status_code=404, detail="Driver not found")
    return updated_driver

# --- CAB ROUTES ---
@app.get("/api/cabs", response_model=List[schemas.CabResponse])
def read_cabs(db: Session = Depends(database.get_db)):
    return crud.get_cabs(db)

@app.get("/api/cabs/available", response_model=List[schemas.CabResponse])
def read_available_cabs(cab_type: Optional[str] = None, db: Session =
Depends(database.get_db)):
    return crud.get_available_cabs(db, cab_type=cab_type)

@app.post("/api/cabs", response_model=schemas.CabResponse)
def create_new_cab(cab: schemas.CabCreate, db: Session = Depends(database.get_db),
current_user: models.User = Depends(auth.require_admin)):
    return crud.create_cab(db, cab)

# --- BOOKING ROUTES ---
@app.post("/api/bookings", response_model=schemas.BookingResponse)
def create_booking_route(booking: schemas.BookingCreate, db: Session =
Depends(database.get_db), current_user: models.User = Depends(auth.get_current_user)):
    return crud.create_booking(db, booking, user_id=current_user.id)

@app.get("/api/bookings/my", response_model=List[schemas.BookingResponse])
def read_my_bookings(db: Session = Depends(database.get_db), current_user: models.User =
Depends(auth.get_current_user)):
    return crud.get_user_bookings(db, user_id=current_user.id)

@app.get("/api/bookings/all", response_model=List[schemas.BookingResponse])
def read_all_bookings(db: Session = Depends(database.get_db), current_user: models.User =
Depends(auth.require_admin)):
    return crud.get_all_bookings(db)

@app.put("/api/bookings/{booking_id}/status", response_model=schemas.BookingResponse)
def update_booking_status_route(booking_id: int, status_update: schemas.BookingStatusUpdate,
db: Session = Depends(database.get_db)):
    updated = crud.update_booking_status(db, booking_id, status_update.booking_status)
    if not updated:
        raise HTTPException(status_code=404, detail="Booking not found")
    return updated

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

Explanation

- `models.Base.metadata.create_all(bind=database.engine)` creates the tables on start-up if they are missing.
- `FastAPI(title=..., description=..., version=...)` sets the information shown in the automatic documentation at `/docs`.
- `CORSMiddleware` allows the front end running on another port to call the API.
- The **authentication routes** `register`, `login` and `me` implement sign-up (duplicate e-mail gives HTTP 400), sign-in (wrong credentials give HTTP 401) and profile lookup.

- The **user routes** are protected by `Depends(auth.require_admin)`; only an administrator can list or delete users.
- The **driver and cab routes** allow everybody to read the lists, while creating a driver or a cab needs the admin role.
- The **booking routes** use `Depends(auth.get_current_user)`, so `create_booking_route()` receives the logged-in user and stores `user_id` from the token, not from the request body. This prevents a user from booking in the name of somebody else.
- `response_model` on every route ensures that the response is filtered and documented according to the schema.
- The last block starts Uvicorn when the file is executed directly.

14.7 Back End: requirements.txt

```
fastapi>=0.104.0
uvicorn[standard]>=0.23.2
sqlalchemy>=2.0.22
pydantic>=2.4.2
pydantic-settings>=2.0.3
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
python-multipart>=0.0.6
```

The file lists the packages that must be installed with `pip install -r requirements.txt`. It was explained in Chapter 8.

14.8 Front End: script.js

`script.js` is the JavaScript engine shared by all HTML pages. It talks to the API using the Fetch API.

```
// AutoCab System Frontend JavaScript Engine
const API_BASE_URL = 'http://localhost:8000/api';

// Helper: Get Auth Token
function getToken() {
  return localStorage.getItem('autocab_token');
}

// Helper: Get Logged In User
function getUser() {
  const userStr = localStorage.getItem('autocab_user');
  return userStr ? JSON.parse(userStr) : null;
}

// Handle User Registration
async function handleRegister(event) {
  event.preventDefault();
  const name = document.getElementById('reg-name').value;
  const email = document.getElementById('reg-email').value;
  const phone = document.getElementById('reg-phone').value;
  const password = document.getElementById('reg-password').value;
  const role = document.getElementById('reg-role').value;

  try {
    const res = await fetch(`${API_BASE_URL}/auth/register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, email, phone, password, role })
    });
    const data = await res.json();
    if (res.ok) {
      alert('Registration successful! Please login.');
      window.location.href = 'login.html';
    } else {
      alert(data.detail || 'Registration failed');
    }
  } catch (err) {
    console.error(err);
    alert('Server connection error.');
  }
}

// Handle User Login
async function handleLogin(event) {
  event.preventDefault();
  const email = document.getElementById('login-email').value;
  const password = document.getElementById('login-password').value;

  try {
    const res = await fetch(`${API_BASE_URL}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });
    const data = await res.json();
    if (res.ok) {
      localStorage.setItem('autocab_token', data.access_token);
      localStorage.setItem('autocab_user', JSON.stringify(data.user));
      alert(`Welcome back, ${data.user.name}!`);
      window.location.href = 'dashboard.html';
    } else {
      alert(data.detail || 'Invalid login credentials');
    }
  } catch (err) {
    console.error(err);
    alert('Server connection error.');
  }
}

// Logout
function logout() {
  localStorage.removeItem('autocab_token');
```

```
  localStorage.removeItem('autocab_user');
  window.location.href = 'login.html';
}

// Fetch and display available cabs for booking
async function loadAvailableCabs() {
  const cabType = document.getElementById('cab-type-filter')?.value || '';
  const container = document.getElementById('cabs-container');
  if (!container) return;

  try {
    const res = await fetch(`${API_BASE_URL}/cabs/available?cab_type=${cabType}`);
    const cabs = await res.json();
    container.innerHTML = cabs.map(cab => `
      <div class="card">
        <h3>${cab.cab_type} Cab</h3>
        <p><strong>Cab Number:</strong> ${cab.cab_number}</p>
        <p><strong>Status:</strong> <span class="badge
badge-success">${cab.availability}</span></p>
        <button class="btn" style="margin-top: 1rem;" onclick="selectCabForBooking(${cab.id},
'${cab.cab_type}')">Book This Cab</button>
      </div>
    `).join('');
  } catch (err) {
    container.innerHTML = '<p>Error loading cabs. Ensure backend server is running.</p>';
  }
}

// Book a Cab
async function handleBooking(event) {
  event.preventDefault();
  const cabId = document.getElementById('selected-cab-id').value;
  const pickup = document.getElementById('pickup-location').value;
  const destination = document.getElementById('destination').value;
  const fare = parseFloat(document.getElementById('calculated-fare').value);

  const token = getToken();
  if (!token) {
    alert('Please login first to book a cab.');
    window.location.href = 'login.html';
    return;
  }

  try {
    const res = await fetch(`${API_BASE_URL}/bookings`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      body: JSON.stringify({
        cab_id: parseInt(cabId),
        pickup_location: pickup,
        destination: destination,
        fare: fare
      })
    });
    if (res.ok) {
      alert('Cab booked successfully!');
      window.location.href = 'dashboard.html';
    } else {
      const err = await res.json();
      alert(err.detail || 'Booking failed');
    }
  } catch (err) {
    alert('Error connecting to backend.');
  }
}
```

Explanation

- `API_BASE_URL` holds the address of the back end; changing it is enough when the server is deployed elsewhere.
- `getToken()` and `getUser()` read the login information that was saved in localStorage after a successful login.

- `handleRegister()` and `handleLogin()` call `event.preventDefault()` to stop the default form submission, send JSON with `fetch()` and show success or error messages. On login the token and user are stored in localStorage.
- `logout()` removes both items from localStorage and redirects to the login page.
- `loadAvailableCabs()` requests `/cabs/available` with the selected category and builds a card for each cab using a template string. If the server is not running, an error message is displayed instead.
- `handleBooking()` first checks that a token exists, then posts the cab ID, pickup, destination and fare with the `Authorization: Bearer` header. On success the user is moved to the dashboard.

14.9 Front End: index.html (Home Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AutoCab System - Smart Cab Booking Platform</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <a href="index.html" class="logo">🚖 AutoCab System</a>
    <ul class="nav-links">
      <li><a href="index.html" class="active">Home</a></li>
      <li><a href="booking.html">Book Cab</a></li>
      <li><a href="dashboard.html">Dashboard</a></li>
      <li><a href="login.html" class="btn">Login</a></li>
      <li><a href="register.html" class="btn btn-secondary">Register</a></li>
    </ul>
  </nav>

  <div class="container">
    <section class="hero">
      <h1>Fast, Reliable & Affordable Rides</h1>
      <p>Book rides in seconds, manage your fleet effortlessly, and enjoy seamless transport
services with AutoCab.</p>
      <div>
        <a href="booking.html" class="btn" style="font-size: 1.1rem; padding: 0.8rem
1.8rem;">Book a Ride Now</a>
      </div>
    </section>

    <h2 style="margin-bottom: 1.5rem; text-align: center;">Our Vehicle Categories</h2>
    <div class="grid">
      <div class="card">
        <h3>🚗 Sedan</h3>
        <p>Comfortable 4-seater cars for everyday city commutes with economical fares.</p>
      </div>
      <div class="card">
        <h3>🚙 SUV</h3>
        <p>Spacious 6-seater vehicles ideal for family trips and heavy luggage transfers.</p>
      </div>
      <div class="card">
        <h3>✨ Luxury</h3>
        <p>Executive premium rides for business travels and VIP transportation.</p>
      </div>
      <div class="card">
        <h3>⚡ Electric EV</h3>
        <p>Eco-friendly zero-emission electric vehicles for green urban mobility.</p>
      </div>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

The home page uses a navigation bar, a hero section and a grid of four category cards. It loads `style.css` for the design. All the cards are static HTML, so the page works even when the server is off.

14.10 Front End: login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - AutoCab System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <a href="index.html" class="logo">🚖 AutoCab System</a>
    <ul class="nav-links">
      <li><a href="index.html">Home</a></li>
      <li><a href="booking.html">Book Cab</a></li>
      <li><a href="register.html" class="btn btn-secondary">Register</a></li>
    </ul>
  </nav>

  <div class="container">
    <div class="card form-card">
      <h2 style="text-align: center; margin-bottom: 1.5rem;">Account Login</h2>
      <form onsubmit="handleLogin(event)">
        <div class="form-group">
          <label>Email Address</label>
          <input type="email" id="login-email" class="form-control" placeholder="e.g.
user@autocab.com" required>
        </div>
        <div class="form-group">
          <label>Password</label>
          <input type="password" id="login-password" class="form-control"
placeholder="••••••••" required>
        </div>
        <button type="submit" class="btn" style="width: 100%;">Sign In</button>
      </form>
      <p style="text-align: center; margin-top: 1rem; color: #64748b;">
        Don't have an account? <a href="register.html">Register here</a>
      </p>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

The login form uses `onsubmit="handleLogin(event)"`. The input type `email` provides basic browser validation and `required` prevents empty submission.

14.11 Front End: register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Registration - AutoCab System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <a href="index.html" class="logo">🚖 AutoCab System</a>
    <ul class="nav-links">
      <li><a href="index.html">Home</a></li>
      <li><a href="login.html" class="btn">Login</a></li>
    </ul>
  </nav>

  <div class="container">
    <div class="card form-card">
      <h2 style="text-align: center; margin-bottom: 1.5rem;">Create an Account</h2>
      <form onsubmit="handleRegister(event)">
        <div class="form-group">
          <label>Full Name</label>
          <input type="text" id="reg-name" class="form-control" placeholder="e.g. John Doe"
required>
        </div>
        <div class="form-group">
          <label>Email Address</label>
          <input type="email" id="reg-email" class="form-control" placeholder="e.g.
john@example.com" required>
        </div>
        <div class="form-group">
          <label>Phone Number</label>
          <input type="tel" id="reg-phone" class="form-control" placeholder="+1-555-0199"
required>
        </div>
        <div class="form-group">
          <label>Password</label>
          <input type="password" id="reg-password" class="form-control" placeholder="••••••••"
required>
        </div>
        <div class="form-group">
          <label>Role</label>
          <select id="reg-role" class="form-control">
            <option value="user">Passenger / User</option>
            <option value="driver">Cab Driver</option>
            <option value="admin">Fleet Admin</option>
          </select>
        </div>
        <button type="submit" class="btn" style="width: 100%;">Create Account</button>
      </form>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

The registration form collects the six pieces of information needed by the `UserCreate` schema. The role drop-down offers passenger, driver and admin.

14.12 Front End: booking.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Book a Cab - AutoCab System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body onload="loadAvailableCabs()">
  <nav class="navbar">
    <a href="index.html" class="logo">🚖 AutoCab System</a>
    <ul class="nav-links">
      <li><a href="index.html">Home</a></li>
      <li><a href="booking.html" class="active">Book Cab</a></li>
      <li><a href="dashboard.html">Dashboard</a></li>
    </ul>
  </nav>

  <div class="container">
    <h2 style="margin-bottom: 1.5rem;">Book Your Ride</h2>

    <div style="display: grid; grid-template-columns: 1fr 2fr; gap: 2rem;">
      <!-- Booking Form -->
      <div class="card">
        <h3>Trip Details</h3>
        <form onsubmit="handleBooking(event)">
          <input type="hidden" id="selected-cab-id" value="1">
          <div class="form-group">
            <label>Pickup Location</label>
            <input type="text" id="pickup-location" class="form-control" placeholder="e.g.
Central Station" required>
          </div>
          <div class="form-group">
            <label>Destination</label>
            <input type="text" id="destination" class="form-control" placeholder="e.g. Airport
Terminal 1" required>
          </div>
          <div class="form-group">
            <label>Estimated Fare (₹)</label>
            <input type="number" id="calculated-fare" class="form-control" value="250.00"
step="10" readonly>
          </div>
          <button type="submit" class="btn" style="width: 100%;">Confirm Booking</button>
        </form>
      </div>

      <!-- Available Cabs List -->
      <div>
        <div style="display: flex; justify-content: space-between; align-items: center;
margin-bottom: 1rem;">
          <h3>Available Cabs</h3>
          <select id="cab-type-filter" class="form-control" style="width: auto;"
onchange="loadAvailableCabs()">
            <option value="">All Categories</option>
            <option value="Sedan">Sedan</option>
            <option value="SUV">SUV</option>
            <option value="Luxury">Luxury</option>
            <option value="EV">EV</option>
          </select>
        </div>
        <div id="cabs-container" class="grid">
          <!-- Dynamically populated cabs -->
        </div>
      </div>
    </div>
  </div>

  <script src="script.js"></script>
  <script>
    function selectCabForBooking(cabId, cabType) {
      document.getElementById('selected-cab-id').value = cabId;
      alert(`Selected Cab #${cabId} (${cabType}). Fill in pickup & destination to complete
booking.`);
    }
  </script>
</body>
</html>
```

The booking page uses `body onload="loadAvailableCabs()"` to fill the cab list as soon as the page opens. A hidden input `selected-cab-id` stores the cab chosen by the user through the small inline function `selectCabForBooking()`. The category drop-down calls `loadAvailableCabs()` on every change, which reloads the list from the server with the new filter.

14.13 Front End: dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard - AutoCab System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="navbar">
    <a href="index.html" class="logo">🚖 AutoCab System</a>
    <ul class="nav-links">
      <li><a href="index.html">Home</a></li>
      <li><a href="booking.html">Book Cab</a></li>
      <li><a href="dashboard.html" class="active">Dashboard</a></li>
      <li><button onclick="logout()" class="btn btn-secondary">Sign Out</button></li>
    </ul>
  </nav>

  <div class="container">
    <div id="user-info-card" class="card" style="margin-bottom: 2rem;">
      <h2>Welcome back to AutoCab Dashboard</h2>
      <p id="user-role-badge" style="margin-top: 0.5rem; color: #64748b;">Loading user
details...</p>
    </div>

    <!-- User Section -->
    <div class="card">
      <h3>My Recent Bookings</h3>
      <div class="table-responsive">
        <table>
          <thead>
            <tr>
              <th>Booking ID</th>
              <th>Pickup</th>
              <th>Destination</th>
              <th>Fare</th>
              <th>Status</th>
            </tr>
          </thead>
          <tbody id="user-bookings-table">
            <tr>
              <td colspan="5" style="text-align: center; color: #64748b;">No recent bookings
found.</td>
            </tr>
          </tbody>
        </table>
      </div>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

The dashboard contains a welcome card and a table with the id `user-bookings-table`, which is filled with the booking history of the logged-in user. The Sign Out button calls `logout()`.

14.14 Front End: style.css

```
/* Modern AutoCab System Styles */
:root {
  --primary: #2563eb;
  --primary-hover: #1d4ed8;
  --dark: #0f172a;
  --card-bg: #ffffff;
  --border: #e2e8f0;
  --text-primary: #1e293b;
  --text-secondary: #64748b;
  --success: #10b981;
  --warning: #f59e0b;
  --danger: #ef4444;
  --radius: 12px;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
}

body {
  background-color: #f8fafc;
  color: var(--text-primary);
  line-height: 1.6;
}

/* Navbar */
.navbar {
  background: var(--dark);
  color: white;
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

.navbar .logo {
  font-size: 1.5rem;
  font-weight: 700;
  color: #38bdf8;
  text-decoration: none;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.nav-links {
  display: flex;
  gap: 1.5rem;
  align-items: center;
  list-style: none;
}

.nav-links a {
  color: #94a3b8;
  text-decoration: none;
  font-weight: 500;
  transition: color 0.2s;
}

.nav-links a:hover, .nav-links a.active {
  color: #ffffff;
}

.btn {
  background: var(--primary);
  color: white;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: var(--radius);
  font-weight: 600;
  cursor: pointer;
  text-decoration: none;
  display: inline-block;
  transition: all 0.2s;
```

```
}

.btn:hover {
  background: var(--primary-hover);
  transform: translateY(-1px);
}

.btn-secondary {
  background: #e2e8f0;
  color: #334155;
}

.btn-secondary:hover {
  background: #cbd5e1;
}

.btn-danger {
  background: var(--danger);
  color: white;
}

/* Container & Layout */
.container {
  max-width: 1200px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}

/* Hero Section */
.hero {
  text-align: center;
  padding: 4rem 1rem;
  background: linear-gradient(135deg, #1e293b 0%, #0f172a 100%);
  color: white;
  border-radius: var(--radius);
  margin-bottom: 2rem;
}

.hero h1 {
  font-size: 3rem;
  margin-bottom: 1rem;
  font-weight: 800;
}

.hero p {
  font-size: 1.2rem;
  color: #94a3b8;
  max-width: 600px;
  margin: 0 auto 2rem;
}

/* Cards & Grid */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 1.5rem;
}

.card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.5rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.02);
  transition: transform 0.2s, box-shadow 0.2s;
}

.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.05);
}

/* Form Styles */
.form-card {
  max-width: 450px;
  margin: 3rem auto;
}

.form-group {
  margin-bottom: 1.25rem;
}
```

```
.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.5rem;
  color: #334155;
}

.form-control {
  width: 100%;
  padding: 0.75rem 1rem;
  border: 1px solid var(--border);
  border-radius: var(--radius);
  font-size: 1rem;
  outline: none;
  transition: border-color 0.2s;
}

.form-control:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.1);
}

/* Status Badges */
.badge {
  padding: 0.25rem 0.75rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 600;
}

.badge-success { background: #d1fae5; color: #065f46; }
.badge-warning { background: #fef3c7; color: #92400e; }
.badge-danger { background: #fee2e2; color: #991b1b; }

/* Table Styles */
.table-responsive {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 1rem;
}

th, td {
  padding: 0.75rem 1rem;
  text-align: left;
  border-bottom: 1px solid var(--border);
}

th {
  background-color: #f1f5f9;
  font-weight: 600;
  color: #475569;
}
```

The stylesheet defines CSS variables in `:root` for the colour scheme, a reset, the navigation bar, hero section, cards, grid, forms, buttons, tables and status badges. Using variables allows the theme to be changed from one place. The grid uses `repeat(auto-fit, minmax(...))` so that cards re-arrange themselves on small screens.

14.15 React Prototype: types.ts

The TypeScript interfaces describe the same entities as the database tables, with the extra fields (for example `fareRatePerKm` and `distanceKm`) used by the dashboards.

```
export type Role = 'user' | 'driver' | 'admin';

export interface User {
  id: number;
  name: string;
  email: string;
  phone: string;
  role: Role;
  createdAt?: string;
}

export interface Driver {
  id: number;
  name: string;
  phone: string;
  licenseNumber: string;
  status: 'Available' | 'Busy' | 'Offline';
  rating?: number;
  totalRides?: number;
}

export interface Cab {
  id: number;
  driverId: number | null;
  driverName?: string;
  cabNumber: string;
  cabType: 'Sedan' | 'SUV' | 'Luxury' | 'EV';
  availability: 'Available' | 'On Ride' | 'Maintenance';
  fareRatePerKm: number;
}

export interface Booking {
  id: number;
  userId: number;
  userName?: string;
  userPhone?: string;
  cabId: number;
  cabNumber?: string;
  cabType?: string;
  driverName?: string;
  pickupLocation: string;
  destination: string;
  fare: number;
  distanceKm: number;
  bookingStatus: 'Pending' | 'Accepted' | 'In Progress' | 'Completed' | 'Cancelled';
  bookingDate: string;
}

export interface SystemStats {
  totalUsers: number;
  totalDrivers: number;
  totalCabs: number;
  availableCabs: number;
  totalBookings: number;
  completedRides: number;
  totalRevenue: number;
}

export interface AuthState {
  user: User | null;
  token: string | null;
}
```

Using union types such as `'Pending' | 'Accepted' | 'In Progress' | 'Completed' | 'Cancelled'` means that the compiler rejects any status that is not part of the life cycle.

14.16 React Prototype: Fare Calculation in BookingPage.tsx

```
import React, { useState } from 'react';
import { Car, MapPin, DollarSign, Clock, CheckCircle2, Shield, AlertCircle, ArrowRight } from
'lucide-react';
import { Cab, User } from '../types';

interface BookingPageProps {
  currentUser: User | null;
  cabs: Cab[];
  onBookRide: (bookingData: {
    cabId: number;
    pickupLocation: string;
    destination: string;
    fare: number;
    distanceKm: number;
  }) => Promise<void>;
  onOpenAuth: () => void;
}

export const BookingPage: React.FC<BookingPageProps> = ({
  currentUser,
  cabs,
  onBookRide,
  onOpenAuth
}) => {
  const [pickup, setPickup] = useState('');
  const [destination, setDestination] = useState('');
  const [selectedCabType, setSelectedCabType] = useState<string>('All');
  const [selectedCabId, setSelectedCabId] = useState<number | null>(null);
  const [distanceKm, setDistanceKm] = useState(14);
  const [loading, setLoading] = useState(false);
  const [successModal, setSuccessModal] = useState(false);
  const [errorMsg, setErrorMsg] = useState('');

  // Filter available cabs
  const availableCabs = cabs.filter(c => {
    const isAvail = c.availability === 'Available';
    if (selectedCabType === 'All') return isAvail;
    return isAvail && c.cabType === selectedCabType;
  });

  const selectedCab = cabs.find(c => c.id === selectedCabId) || availableCabs[0];
  const currentRate = selectedCab ? selectedCab.fareRatePerKm : 18.00;
  const calculatedFare = parseFloat((distanceKm * currentRate + 30.0).toFixed(2));

  const handleSubmitBooking = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!currentUser) {
      onOpenAuth();
      return;
    }

    if (!pickup.trim() || !destination.trim()) {
      setErrorMsg('Please enter both pickup and destination locations.');
      return;
    }

    const cabToBook = selectedCab || availableCabs[0];
    if (!cabToBook) {
      setErrorMsg('No available cab selected.');
      return;
    }

    try {
      setLoading(true);
      setErrorMsg('');
      await onBookRide({
        cabId: cabToBook.id,
        pickupLocation: pickup,
        destination,
        fare: calculatedFare,
        distanceKm
      });
      setSuccessModal(true);
      setPickup('');
      setDestination('');
    } catch (err: any) {
      setErrorMsg(err.message || 'Failed to place cab booking.');
    } finally {
```

```
      setLoading(false);
    }
  };
```

Explanation

- The component receives the list of cabs and the callback `onBookRide` as props.
- `availableCabs` filters the cabs by availability and by the selected category.
- `calculatedFare = distanceKm * currentRate + 30.0` is the fare formula: distance times the per-km rate of the selected cab plus a base fare of ₹30.
- `handleSubmitBooking()` stops the default form action, asks a non-logged-in user to sign in, checks that pickup and destination are filled, and calls `onBookRide()` with the calculated fare. Errors are shown in a red message box and a success dialog is shown after booking.

14.17 React Prototype: Status Rules in server.ts

The Express server of the prototype implements the same booking rules as the FastAPI back end. The part that creates a booking and updates its status is shown below.

```
  app.post('/api/bookings', (req: any, res) => {
    const { cab_id, pickup_location, destination, fare, distanceKm } = req.body;

    if (!cab_id || !pickup_location || !destination) {
      return res.status(400).json({ detail: 'Cab ID, pickup, and destination are
required.' });
    }

    const cab = cabs.find(c => c.id === parseInt(cab_id));
    if (!cab) {
      return res.status(404).json({ detail: 'Selected cab not found.' });
    }

    const driver = drivers.find(d => d.id === cab.driverId);
    const userId = req.user ? req.user.id : 4;
    const currentUser = users.find(u => u.id === userId) || users[3];

    const computedFare = fare ? parseFloat(fare) : 250.00;
    const dist = distanceKm ? parseFloat(distanceKm) : 10.5;

    const newBooking: Booking = {
      id: bookings.length ? Math.max(...bookings.map(b => b.id)) + 1 : 101,
      userId: currentUser.id,
      userName: currentUser.name,
      userPhone: currentUser.phone,
      cabId: cab.id,
      cabNumber: cab.cabNumber,
      cabType: cab.cabType,
      driverName: driver ? driver.name : 'John Smith',
      pickupLocation: pickup_location,
      destination,
      fare: computedFare,
      distanceKm: dist,
      bookingStatus: 'Pending',
      bookingDate: new Date().toISOString().replace('T', ' ').substring(0, 19)
    };

    bookings.unshift(newBooking);

    // Update Cab & Driver status
    cab.availability = 'On Ride';
    if (driver) {
      driver.status = 'Busy';
    }

    res.status(201).json(newBooking);
  });

  app.put('/api/bookings/:id/status', (req, res) => {
    const id = parseInt(req.params.id);
    const { booking_status } = req.body;

    const booking = bookings.find(b => b.id === id);
    if (!booking) {
      return res.status(404).json({ detail: 'Booking not found' });
    }

    booking.bookingStatus = booking_status;

    // Release Cab & Driver if Completed or Cancelled
    if (booking_status === 'Completed' || booking_status === 'Cancelled') {
      const cab = cabs.find(c => c.id === booking.cabId);
      if (cab) {
        cab.availability = 'Available';
        if (cab.driverId) {
          const driver = drivers.find(d => d.id === cab.driverId);
          if (driver) {
            driver.status = 'Available';
            if (booking_status === 'Completed') {
              driver.totalRides = (driver.totalRides || 0) + 1;
            }
          }
```

```
        }
      }
    }

    res.json(booking);
});
```

Explanation

- When a booking is created, the cab is set to "On Ride" and its driver to "Busy".
- In the status route, the values "Completed" or "Cancelled" release the cab and driver; a completed ride also increases the driver's `totalRides` counter.
- The booking is placed at the start of the list with `unshift()` so that the newest booking appears first.

15. SOFTWARE TESTING

15.1 Introduction to Testing

Software testing is the process of running the software with chosen inputs to find defects and to check that it behaves as required. For AutoCab, testing is planned at four levels: unit testing of individual functions, integration testing of the layers working together, system testing of complete user scenarios, and security testing of authentication and authorisation. FastAPI makes API testing easy because the interactive Swagger page at `/docs` can send requests with sample data and show the response code and body.

15.2 Testing Strategy

Table 17: Levels of testing

Level	What is tested	Method
Unit testing	Functions such as get_password_hash, verify_password, create_access_token, get_available_cabs, update_booking_status	Calling the function with sample data and checking the return value
Integration testing	Route + schema + CRUD + database working together	Swagger UI or an HTTP client sending requests to the running server
System testing	Complete flows: register → login → book → complete	Manual testing through the browser pages
Security testing	Token checking, role restriction, duplicate e-mail	Sending requests without token, with wrong token and with a non-admin token
Compatibility testing	Pages on different browsers and screen sizes	Chrome, Edge and Firefox; desktop and mobile view

15.3 Test Cases – Authentication

Table 18: Authentication test cases

ID	Test case	Input	Expected result	Status
TC01	Register with valid data	Name, new e-mail, phone, password, role	HTTP 200; user created with id; password not shown	
TC02	Register with existing e-mail	E-mail already in database	HTTP 400 "Email address is already registered."	
TC03	Register with invalid e-mail	abc.com	HTTP 422 validation error	
TC04	Login with correct credentials	user@autocab.com / user123	HTTP 200; access_token and user returned	
TC05	Login with wrong password	Correct e-mail, wrong password	HTTP 401 "Incorrect email or password"	
TC06	Login with unknown e-mail	unknown@x.com	HTTP 401	
TC07	Access /api/auth/me with token	Valid Bearer token	HTTP 200; profile returned	
TC08	Access /api/auth/me without token	No header	HTTP 401 "Not authenticated"	
TC09	Access with tampered token	Token with changed characters	HTTP 401 "Could not validate credentials"	

15.4 Test Cases – Cabs and Bookings

Table 19: Cab and booking test cases

ID	Test case	Input	Expected result	Status
TC10	View available cabs	GET /api/cabs/available	Only cabs with availability "Available" are listed	
TC11	Filter by category	cab_type = SUV	Only available SUV cabs are listed	

ID	Test case	Input	Expected result	Status
TC12	Create booking when logged in	cab_id, pickup, destination, fare + token	HTTP 200; status "Pending"; cab becomes "On Ride"	
TC13	Create booking without login	Same data, no token	HTTP 401; no booking created	
TC14	Booked cab disappears from list	GET available cabs after TC12	The booked cab is not in the list	
TC15	View my bookings	GET /api/bookings/my with token	Only the logged-in user's bookings, newest first	
TC16	Complete a booking	PUT status = Completed	Status updated; cab becomes "Available" again	
TC17	Cancel a booking	PUT status = Cancelled	Status updated; cab becomes "Available" again	
TC18	Update status of unknown booking	booking_id = 9999	HTTP 404 "Booking not found"	

15.5 Test Cases – Admin Functions

Table 20: Admin test cases

ID	Test case	Input	Expected result	Status
TC19	List users as admin	Admin token	HTTP 200; list of all users	
TC20	List users as passenger	Passenger token	HTTP 403 "Operation restricted to admin users only."	
TC21	Add driver as admin	Name, phone, unique license number	HTTP 200; driver created	
TC22	Add cab as admin	cab_number, cab_type	HTTP 200; cab created with availability "Available"	

ID	Test case	Input	Expected result	Status
TC23	Add cab with duplicate number	Existing cab_number	Request rejected because cab_number is unique	
TC24	View all bookings as admin	Admin token	HTTP 200; all bookings of all users	
TC25	Delete user as admin	Valid user_id	HTTP 200 "User deleted successfully"	
TC26	Change driver status	status = Offline	Driver status updated	

15.6 Test Cases – User Interface

Table 21: Front-end test cases

ID	Test case	Expected result	Status
TC27	Open home page	Hero text and four category cards are displayed	
TC28	Submit login form with empty fields	Browser blocks submission (required fields)	
TC29	Open booking page	Available cabs are loaded automatically	
TC30	Change category drop-down	Cab list reloads with the chosen category	
TC31	Book a cab without login	Alert "Please login first to book a cab." and redirect to login page	
TC32	Stop back-end server and open booking page	Message "Error loading cabs. Ensure backend server is running."	
TC33	Click Sign Out	Token removed from localStorage; login page opens	
TC34	View pages on a mobile-width screen	Layout re-arranges without horizontal scrolling	

The Status column is filled after each test case has been executed on the running system (Pass / Fail), and any failed case is corrected and tested again (regression testing).

15.7 Sample API Test using Swagger UI

To test the booking flow manually: (1) start the server and open `http://localhost:8000/docs`; (2) run `POST /api/auth/register` to create a passenger; (3) run `POST /api/auth/login` and copy the `access_token`; (4) press the **Authorize** button and paste the token; (5) run `GET /api/cabs/available` and note a cab ID; (6) run `POST /api/bookings` with the body `{"cab_id": 1, "pickup_location": "Central Station", "destination": "Airport", "fare": 357.6}`; (7) run `GET /api/cabs/available` again and confirm that the cab is no longer listed; (8) run `PUT /api/bookings/{id}/status` with `{"booking_status": "Completed"}` and confirm that the cab is available again.

15.8 Observations and Points for Improvement

During testing and code review the following points were noted. They do not stop the system from working but are important for a production version and are listed here honestly as part of the evaluation of the project.

- The routes `PUT /api/drivers/{id}/status` and `PUT /api/bookings/{id}/status` currently do not require a token. In production they should check that the caller is the assigned driver or an admin.
- The registration form allows the user to choose the "admin" role. In production, admin accounts should be created only by an existing admin.
- The JWT secret key is stored in the source code and CORS allows all origins; both should be moved to environment settings.
- The estimated fare in the plain HTML booking page is a fixed default value; the distance-based calculation is implemented in the React prototype.
- `datetime.utcnow` stores UTC time; the front end can convert it to local time for display.

16. CONCLUSION

The AutoCab Booking System has been developed as a complete web-based solution for cab booking and fleet management. It replaces telephone calls and paper registers with a secure, centralised application in which passengers, drivers and administrators use the same up-to-date data. The system fulfils the objectives set at the beginning of the project: a multi-role portal, secure authentication with JWT and bcrypt, vehicle categories, fare estimation, a complete booking life cycle, and an administrative interface for users, drivers, cabs and bookings.

The most important design decision of the project is the automatic synchronisation of booking status and cab availability. Because the cab is marked "On Ride" at the moment of booking and released when the trip is completed or cancelled, over-booking is eliminated without any manual work. The layered architecture (routes, schemas, CRUD functions, models and database) keeps the code organised and easy to maintain, and the use of open-source tools keeps the cost very low for small transport businesses.

During the development the author gained practical experience in REST API design with FastAPI, object-relational mapping with SQLAlchemy, token-based security, database design with primary and foreign keys, front-end development with HTML, CSS, JavaScript and React, and in preparing system documentation such as DFD, ER and flow diagrams.

16.1 Future Scope

- **GPS integration:** real-time tracking of cabs on an interactive map using the Google Maps API, and automatic calculation of distance and fare from the map route.
- **Online payments:** integration of payment gateways such as Razorpay for cashless payment and automatic receipts.
- **AI analytics:** use of machine learning to predict peak hours and to distribute cabs according to demand.
- **Mobile application:** native Android and iOS applications for better accessibility.
- **Notifications:** SMS and e-mail alerts for booking confirmation and driver arrival.
- **Ratings and reviews:** passenger feedback for drivers and automatic driver assignment based on distance and rating.
- **Advanced security:** refresh tokens, environment-based secrets and role checks on all status-changing routes.

17. BIBLIOGRAPHY / REFERENCES

1. Tiangolo, S. (2023). FastAPI Documentation. https://fastapi.tiangolo.com
2. Python Software Foundation. Python Official Documentation. https://docs.python.org/3/
3. SQLite Consortium. SQLite Official Documentation. https://www.sqlite.org/docs.html
4. SQLAlchemy Authors. SQLAlchemy Documentation. https://docs.sqlalchemy.org/
5. Pydantic Documentation. https://docs.pydantic.dev/
6. Jones, M., Bradley, J., Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519
7. Mozilla Developer Network. MDN Web Docs – HTML, CSS, JavaScript and Fetch API. https://developer.mozilla.org/
8. React Documentation. https://react.dev/
9. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.

Student Profile:-



Name: Rahul Pal

Student Code: B2492R10400159

University / Department: AKS University, Department of Computer Science

Course / Year: Bachelor of Computer Application (BCA), 5th Semester

Session: 2026

Project Title: AutoCab Booking System

Project Guide: Dr. Pinki Sharma (Assistant Professor)

The student has developed the “AutoCab Booking System” to provide a simple and reliable web-based platform for cab booking and fleet management. Passengers can register, log in, view the cabs that are currently available in the Sedan, SUV, Luxury and EV categories, see the estimated fare and book a ride instantly. Drivers manage their availability and the trips assigned to them, and the administrator manages drivers, cabs, users, bookings and revenue.

Every booking follows a defined life cycle, and the system automatically marks a cab as “On Ride” when it is booked and “Available” again when the trip ends, which prevents over-booking. The project is built with the FastAPI framework of Python and a SQLite database, and it uses JWT authentication, bcrypt password hashing and role-based authorisation to keep user data secure.

Student Profile:-



Name: Balendra Singh

Student Code: B2492R10400158

University / Department: AKS University, Department of Computer Science

Course / Year: Bachelor of Computer Application (BCA), 5th Semester

Session: 2026

Project Title: AutoCab Booking System

Project Guide: Dr. Pinki Sharma (Assistant Professor)

The student has developed the “AutoCab Booking System” to provide a simple and reliable web-based platform for cab booking and fleet management. Passengers can register, log in, view the cabs that are currently available in the Sedan, SUV, Luxury and EV categories, see the estimated fare and book a ride instantly. Drivers manage their availability and the trips assigned to them, and the administrator manages drivers, cabs, users, bookings and revenue.

Every booking follows a defined life cycle, and the system automatically marks a cab as “On Ride” when it is booked and “Available” again when the trip ends, which prevents over-booking. The project is built with the FastAPI framework of Python and a SQLite database, and it uses JWT authentication, bcrypt password hashing and role-based authorisation to keep user data secure.